

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



**INTEGRATING DEVOPS AND NATURAL LANGUAGE PROCESSING
TO STREAMLINE THESIS MANAGEMENT**

By

Dang Nhat Huy - ITITIU20043

Instructed by

Tran Thanh Tung, PhD

A thesis submitted to the School of Computer Science and Engineering in partial
fulfillment of the requirements for the degree of Bachelor of Information
Technology/Computer Science/Computer Engineering

Ho Chi Minh City, Vietnam

2024

INTEGRATING DEVOPS AND NATURAL LANGUAGE PROCESSING TO STREAMLINE THESIS MANAGEMENT

APPROVED BY:

_____,
Nguyen Van A, Ph.D., Chair (*Example*)

**(Type Committee names beneath lines)*

(Typed Committee name here)

(Typed Committee name here)

(Typed Committee name here)

(Typed Committee name here)

THESIS COMMITTEE
(Whichever applies)

ACKNOWLEDGMENTS

I would like to use this opportunity to thank Dr. Tran Thanh Tung from the bottom of my heart. His advice has been crucial throughout my thesis assignment. His insightful advice pushed me to develop my work in a much stronger direction, and I couldn't have done it without him.

I also want to thank the School of Computer Science and Engineering at the International University. The fundamental information covered in the curriculum gives me the confidence to carry out this study. It has been a pleasant experience, and I was able to participate in the Computer Science program and learn from experienced lecturers.

TABLE OF CONTENTS

ACKNOWLEDGMENTS.....	3
TABLE OF CONTENTS.....	4
LIST OF FIGURES.....	7
LIST OF TABLES.....	10
ABSTRACT.....	11
CHAPTER 1.....	12
INTRODUCTION.....	12
1.1. Motivation.....	12
1.2. Problem Statement.....	14
1.3. Scope and Objectives.....	14
1.3.1. Scope.....	14
1.3.2. Objectives.....	15
1.4. Assumptions and Solutions.....	16
1.4.1. Assumptions.....	16
1.4.2. Solutions.....	16
1.5. Structure of thesis.....	17
CHAPTER 2.....	19
RELATED WORK & COMPARISON.....	19
2.1. Related Work.....	19
2.1.1. ChatPDF.....	19
2.1.2. ChatGPT GPT-4.....	22
2.1.3. PDFgear Copilot.....	24
2.2. Comparison.....	28
2.3. Background.....	29
2.3.1. Event-driven architecture (EDA).....	29
2.3.2. DevOps for Scalability and Reliability.....	32
2.3.3. Python Natural Language Processing (NLP) for Thesis Analysis.....	34
2.3.4. Challenges in Thesis Report Management.....	34

CHAPTER 3.....	35
METHODOLOGY.....	35
3.1. Requirement analysis.....	35
3.1.1. Functional requirements.....	35
3.1.2. Non-functional requirements.....	39
3.1.3. Use Case diagram.....	41
3.2. System Design.....	43
3.2.1. System Architecture.....	43
3.2.2. Database design.....	47
3.2.3. CI/CD pipeline.....	49
3.2.4. Cluster Architecture.....	50
3.2.5. Deployment and Orchestration.....	51
3.2.6. Communication and Data Flow.....	52
CHAPTER 4.....	54
IMPLEMENT AND RESULTS.....	54
4.1. Implement.....	54
4.1.1. Implementing Citation Service.....	54
4.1.2. Implementing Table and Figure Detection Service.....	59
4.1.3. Implementing Grammar Service.....	63
4.1.4. Implementing the Kubernetes Cluster.....	65
4.1.5. Implementing Kubernetes Cluster on ArgoCD.....	71
4.1.6. Configure Jenkins.....	72
4.2. Results.....	73
CHAPTER 5.....	83
DISCUSSION AND EVALUATION.....	83
5.1. Discussion.....	83
5.2. Comparison.....	83
5.3. Evaluation.....	84
5.3.1. Pods fail while executing.....	84
5.3.2. Performance testing.....	85

5.3.3. Real-life evaluation.....	88
5.3.4. Conclusion.....	90
CHAPTER 6.....	91
CONCLUSION AND FUTURE WORK.....	91
6.1. Conclusion.....	91
6.2. Future work.....	92
REFERENCES.....	93
APPENDIX.....	95

LIST OF FIGURES

Figure 1. ChatPDF.....	19
Figure 2. ChatPDF analyzing PDF files.....	20
Figure 3. An error attempt on ChatPDF.....	21
Figure 4. ChatGPT conversational chatbot.....	22
Figure 5. ChatGPT GPT-4 interacts with PDF files.....	23
Figure 6. PDFgear Copilot.....	25
Figure 7. PDFgear Copilot dashboard.....	26
Figure 8. PDFgear Copilot feature limitations.....	27
Figure 9. Message flow in Event-Driven Architecture.....	30
Figure 10. Pub/Sub Pattern in Event-Driven Architecture.....	31
Figure 11. DevOps Concept.....	32
Figure 12. Cross-functional model of DevOps.....	33
Figure 13. Use Case diagram.....	41
Figure 15. Simplified system architecture.....	43
Figure 16. Simplified Event-driven Architecture for our proposed system.....	43
Figure 17. Event-driven microservices architecture for our proposed system.....	45
Figure 18. Exchanges in RabbitMQ.....	46
Figure 19. REST database.....	47
Figure 20. Analysis service database.....	48
Figure 21. Simplified CI/CD and Git Workflow Diagram.....	49
Figure 22. CI/CD pipeline with Jenkins, Docker, and ArgoCD.....	49
Figure 23. Kubernetes Cluster Architecture.....	50
Figure 24. DevOps Integration of Kubernetes with Jenkins, Docker, and ArgoCD.....	51
Figure 25. Google Cloud Storage Data Flow.....	53
Figure 26. Citation Service.....	55
Figure 27. Pseudocode of citation processing function.....	56
Figure 28. Pseudocode of citation merging function.....	57
Figure 29. Pseudocode of citation checking function.....	58

Figure 30. Style-checking functions.....	59
Figure 31. Table/Figure Detection Service.....	60
Figure 32. Pseudocode of extract_figure_titles function.....	60
Figure 33. Pseudocode of extract_table_titles function.....	61
Figure 34. Pseudocode of find_figure_mentions and find_table_mentions functions.....	62
Figure 35. Grammar Service.....	63
Figure 36. Pseudocode of check_grammar functions.....	64
Figure 37. A sample Deployment of a service.....	65
Figure 38. ConfigMap to store non-confidential data.....	67
Figure 39. PostgreSQL StatefulSet.....	69
Figure 40. Kubernetes Cluster visualized on ArgoCD.....	71
Figure 41. A stage of the Jenkins pipeline.....	72
Figure 42. Comprehensive Jenkins pipeline.....	73
Figure 43. Login Page.....	73
Figure 44. Submission Page.....	74
Figure 45. Thesis Report Management's dashboard.....	75
Figure 46. Citation service output.....	76
Figure 47. Figure/Table detection service output.....	77
Figure 48. Grammar checking service output.....	78
Figure 49. Example of removing Table of content service.....	79
Figure 50. Removing service in CD pipeline.....	79
Figure 51. Example of adding Table of content service.....	80
Figure 52. Adding service in CD pipeline.....	80
Figure 53. ArgoCD automates update Cluster.....	81
Figure 54. Pods change on the Kubernetes cluster.....	81
Figure 55. Services exposing a network for each pod.....	82
Figure 56. Grafana dashboard.....	82
Figure 57. New pods created right after the current pods deleted.....	84
Figure 58. First benchmark test with 100 users.....	85
Figure 59. Second benchmark test with 1000 users.....	86

Figure 60. Third benchmark test with 100000 users.....	87
Figure 61. Kubernetes Node Benchmark: Time and Throughput.....	88
Figure 62. Percentage CPU usage on a worker node handling 10 reports at the same time.....	88
Figure 63. Configure simultaneous requests in jMeter.....	89
Figure 64. Response time vs Request per second.....	89
Figure 65. Sample PDF report.....	90

LIST OF TABLES

Table 1. Differences between free and paid ChatGPT [12].....	24
Table 2. Comparison between existing tools.....	28
Table 3. Functional Requirement: NLP-Enhanced Report Processing.....	35
Table 4. Functional Requirement: Jenkins CI Integration.....	36
Table 5. Functional Requirement: ArgoCD for Kubernetes Deployment.....	36
Table 6. Functional Requirement: Virtual Machine (VM) Server.....	37
Table 7. Functional Requirement: Prometheus and Grafana for Monitoring.....	37
Table 8. Functional Requirement: Data Storage and Retrieval.....	38
Table 9. Non-functional Requirement: Push development container image after building.....	39
Table 10. Non-functional Requirement: Scalable Processing of Concurrent Requests.....	39
Table 11. Non-functional Requirement: Robust Event-Driven Messaging Infrastructure.....	40
Table 12. Non-functional Requirement: Comprehensive Logging and Monitoring.....	40
Table 13. Non-functional Requirement: Secure Data Handling and Access Control.....	41
Table 14. Use Case Description.....	42
Table 15. REST database description.....	48

ABSTRACT

The existing system for handling university thesis reports mainly focuses on simple processes like storing and submitting files but requires more advanced content analysis and grammar detection services. There is a limitation to the above scenario, as students need help to refine their work, and commonly available tools like Turnitin that would make this possible can be accessed only by the faculty. In addition, these advanced tools are expensive and cumbersome to integrate into existing systems and, hence, hard to access. More faculty time and effort may be required during peak submission periods to manually assess complete theses, which could be more efficient with the current methods. The cumbersome and error-prone process always ended with wrong module formatting, citation mistakes, and inconsistent grading. These problems signal the development of a better thesis report management system that represents a higher level of student work and facilitates the educational process.

To cope with those issues, We recommend a new thesis report management system that standardizes content analysis, grammar correction, and format validation, making these tools accessible to students and faculty. The system is expected to help the school develop a system that can be used and will reduce the burden on the lecturers regarding administrative work, making the educational process more efficient and improving the students' work.

Consequently, the idea of thesis report administration is not only beneficial in creating pollutant-free atmospheres by allowing every student to have the right educational gadgets, but it is also helpful in students by giving the teachers quick and positive feedback.

CHAPTER 1

INTRODUCTION

1.1. Motivation

The system for dropping off theses at universities must be updated and anachronistic. Take Blackboard (or Canvas) as an example: it allows you to upload and submit files [1], and that is pretty much it - no sophisticated plagiarism or grammar checks. Turnitin and similar tools can accomplish those, but they require support for students to leverage, and those errors are only caught after the paper is turned in. This paper discusses an approach to integrating Natural Language Processing (NLP) and DevOps in a single system. Using a more organized approach, this attempts to simplify and speed up the thesis for instructors and students alike.

The biggest problem with the current system is that it only does a little. It's just a digital filing cabinet. You can't analyze the content of theses or check for plagiarism automatically [2]. Trying to connect these systems to checkers like Turnitin can be a nightmare; even if you do, teachers are the only ones who can use them. This leaves students in the dark about any issues in their work before they submit it.

In addition to the tech limitations, the current system creates a lot of extra work for teachers. Grading season means wading through hundreds of theses, all by hand. This is labor-intensive and prone to errors. Language, formatting, and citation errors may slip through the gaps, lowering the caliber of the theses and adding time to the grading process.

Building a thesis management system using DevOps concepts would be like having an all-around super-assistant. The core of DevOps is continuous improvement and teamwork. As such, the system would continually improve and adjust to new requirements. Along with boosting collaboration among educators, IT staff, and school administrators, it would ensure the system functions appropriately for everyone involved.

Computers interpret human language, known as natural language processing (NLP). This method may automatically detect plagiarism, pointing out any material that might have been precisely stolen from other sources without giving due acknowledgment [3]. Students could use this to locate and fix these issues before handing in their work. NLP might also check for grammatical and citation errors to ensure they adhere to the proper structure and referencing styles. This would generally raise the standard of theses and spare professors' time from all that laborious mistake-hunting [3].

The system could handle even the busiest times of the year. NLP tools can be optimized to analyze large amounts of text data, giving students feedback quickly. DevOps principles could also automatically adjust the system's capacity based on how many people use it at a given time. This would minimize resource use during slow periods and keep things running smoothly during peak submission times [4].

This new system would cater to the growing desire for personalized learning. Students could leverage NLP features to get feedback on their writing style, clarity, and how well they follow academic formatting. This would empower them to take ownership of their learning and improve their writing skills more effectively.

One of the hallmarks of this system is its flexibility to a university's constantly developing requirements. DevOps practices allow agile change and continual addition of new features as required. This ensures the system stays current and gets done according to the latest education standards.

Even though this modern way of doing things can be an excellent idea, it is crucial to consider two points: cost and security. For the system to work, it is vital to have several system parts connected, and in this context, the security must be strong enough to keep student and teacher data safe. Steps to this end include choosing secure user passwords, data encryption, and regular security audits.

The benefits of implementing a plagiarism checker like Turnitin on a long-term scale would be much higher than the likely initial expenses. Automating these processes would lead to significant time savings for teachers, and paper grading could be done with lower costs. In addition, the improved thesis management process might result in the release of some funds that could otherwise be allocated elsewhere in the field of education.

An NLP and DevOps computer-assisted report management system would drastically change processing and grading [5]. The application can deploy intelligent automation and analysis, allowing educators and students more time to participate in the activity and receive advice and support from the implementation. As such, the implementation of a learning project will be more efficient. The new system, however, will lead to better-quality papers for the students, and the teachers can focus on other tasks. Security and expenses are two main aspects that need reasonable consideration, but the long-term ramifications, such as this magic software present for the education community, would be very important [6].

1.2. Problem Statement

At their basis, the management systems for university theses are made for basic operations such as file storage and submission but need to include more advanced features for content analysis and plagiarism detection. This shortcoming leads to the lack of students' empowerment to perform corrections of their writings, as tools like Turnitin that can provide this function are usually available only for teaching staff. Moreover, allocating and setting these implements up in systems already in place is expensive and laborious, restricting their use. Especially at the height of thesis submission periods, the manual review method by faculty members is the most inefficient. This is a long-haul and mistake-prone method that could be the cause of formatting errors, citation mistakes, and non-uniform grading. Such roadblocks make the case for devising a fuller and more efficient thesis report management system that amplifies student submissions and intensifies educational procedures [7].

1.3. Scope and Objectives

1.3.1. Scope

This plan is to build a thesis project management system initially made for the School of Computer Science and Engineering, a department of the International University, a part of Vietnam National University in Ho Chi Minh City. Natural Language Processing (NLP) and DevOps concepts are combined into the system to make the submission and review process easier, which is not a very Yoda-like sentence; professors and students can rely on the system.

One of the main objectives of the update is to amplify the capacities of the existing system to incorporate cutting-edge technologies that guarantee the optimal operation of the system even during high-demand periods in the school calendar. This comprises reliable handling of numerous submissions, a supply of real-time administrative support, and the highest data safety level to prevent privacy breaches. Also, the platform is to decrease the total expenses during its operation, which undoubtedly is a major aspect for educational institutions.

Not only has it been developed for the School of Computer Science and Engineering, but it can also be expanded and changed. NLP and DevOps are a promising combination, and further adaptation of our system to other university fields is potential. It has an excellent architectural design and can be extended to the needs of larger groups while being scaled down for personal use.

System administrators, teachers, and students can access multiple user interfaces to fulfill various needs in academic settings. Whether students upload their theses or professors mark them, they can easily navigate the system because each interface is designed based on their needs and problems.

In conclusion, this strategy is one step toward building a more connected and technologically advanced learning environment, which might benefit the institution and those off campus. Furthermore, it might improve the thesis submission process.

1.3.2. Objectives

The thesis is dedicated to developing a more efficient and supportive submission process. It is characterized by two main objectives: efficiency and empowerment, using DevOps and Natural Language Processing (NLP) as the main tools.

(i) Specifically, the aim will be to reduce the following:

- Automating repeatedly required tasks: NLP automating shedding efforts that are usually time-consuming will finally enable the human brain to allocate all its resources to more complicated affairs such as creative tasks, reading, analyzing, or deducing.
- Advancing operations: We aim to foster orthogonally distributed processing systems instead of the stereotypical greenfield architecture systems, thus automating the process from early to late stage. Automating it will solve the "Show-stoppers"/fragility issues.
- Pressure on the infrastructure: kits and techniques for harnessing system and traffic will not cause your development to lag into obsolescence because of the inevitable high streamlines it will experience when you apply new coding technologies within the deployed software set.

(ii) Moreover, students are to be rendered the faculty tool in fostering the learning process:

- Offering feedback in real-time: Students will receive concise and euphemistic comments on their written work, which will eventually help them become better statutory potential researchers and enhance the final versions of their papers.
- Promoting more innovative communication: My work on the platform makes it very simple for supervisors and advisors of the student population to be effectively treated through probation, the first measure of maintaining the staff's professionalism and the study's success.

- Visualize all processes: Every piece of the thesis writing procedure will be lucid, which will help cultivate trust and decrease the stress people face. The new technology focuses more sharply on supporting students and faculty at all stages than on the stigma of the old system.

In short, this project aims to show that submitting a thesis for college students can be smooth and successful, using innovative technology to help teachers and students at every phase.

1.4. Assumptions and Solutions

1.4.1. Assumptions

We assume that event-driven architecture and microservices are necessary to create a high-quality, flexible thesis management system that can be adapted to changing needs and requirements over time.

We assume that using open-source technology encourages employee collaboration and initiates a culture of innovation.

We assume that this system will be more efficient and agile with the inclusion of DevOps practices. Moreover, DevOps approaches that allow for the organization of the setup breaks, the blend of operational effort, and the execution of the production order at an acceptable first time immediately reduce, therefore, the need for downtime, cut down technical overhead, improve the system performance end-user experience, respectively.

We assume that providing learners with real-time, actionable feedback will have a substantial positive, subliminal effect on the quality of published works.

We assume that it is essential that the system should be expandable and firm during the peak thesis period. When the traffic reaches a level that will overload the system, the software, capable of system logic, creates a virtual machine to handle the users' requests instead. When such a case, even if the system is not overloaded, the VMs can still contain students' activities, and the server can deliver a correct answer to students or teachers promptly.

1.4.2. Solutions

My method comprises DevOps and Natural Language Processing (NLP) and uses NLP to resolve thesis issues.

(i) Choice of Natural Language Processing (NLP) as the solution:

- **Automated Content Review:** The system will automatically check every thesis for syntax, formatting, and grammatical problems using natural language. Students can rectify basic errors in their work before submitting it to an instructor. They can also be sure that they meet the academic standards.
- **Real-Time Feedback:** Natural language processing (NLP) tools will offer real-time suggestions for improving students' writing's coherence, structure, and flow while they work on their theses. Students who get such prompt criticism are making their work better the more it gets to completion.

(ii) Practices of DevOps implementation:

- **The embrace of DevOps** will allow the system to be updated and improved continuously without interruptions in user experiences. It ensures the system accepts user input and keeps abreast of the latest technical developments.
- **Scalability and Flexibility:** DevOps principles will guide the development of a system that can scale up or down based on user demand, which is significant during peak submission times. That means the system will continue operating smoothly, irrespective of the number of users.
- **Enhanced Collaboration Tools:** DevOps involves a shared system development and a management approach that involves the people who have worked on it. For the thesis management system, features like version control and progress tracking are integrated, which allow students and advisors to communicate more effectively throughout the thesis development process.

(iii) Combined Impact:

Combining NLP and DevOps overcomes concrete technical matters and fosters a more dynamic and supportive environment for project management. Students are provided with a platform that helps raise their work quality while reducing their stress, while professors can now operate with a better-organized grading and review process. This all-embracing approach ensures that the software system in the thesis management system serves not just as a submission tool but as the whole help layer for all the users in their educational journey.

1.5. Structure of thesis

The following sections comprise the thesis: Abstract, Introduction, Related Work and Comparison, Methodology, Implementation and Results, Discussion and Evaluation,

Conclusion and Future Work, and References. This thesis is formatted according to the International University - VNUHCMC standard for an academic research paper.

- Abstract: A synopsis of the critical conclusions that lays the groundwork for the remainder.
- Introduction: This section describes the backdrop that inspired the study, defines the problem, and explains the goals and scope of the investigation.
- Related Work and Comparison: This work examines currently in-use applications and contrasts their technologies and approaches with those of this work.
- Methodology: Describes the design and development strategies, including the tools and rationale behind them. It also uses diagrams to help explain the system's requirements and architecture.
- Implementation and Results: This section details the setup and structure of the developed system, demonstrating how it works through various use cases. The outcomes are shown using visuals like images or videos.
- Discussion and Evaluation: Examine the system's efficacy and discuss, based on the implementation's outcomes, what went well and what didn't.
- Conclusion and Future Work: Reviewing the original motivation, summarizing the main conclusions, and discussing the ramifications close the thesis. It also describes prospective project extensions and enhancements.
- References: Lists all the sources cited throughout the research, acknowledging their contribution to the development of the thesis.

CHAPTER 2

RELATED WORK & COMPARISON

2.1. Related Work

Digital document management evolves quickly these days. One such case is the development of software related to the interaction of humans and robots (Conversational AI) and DevOps. Here, we are studying the available systems and techniques that have played a significant role in this area, giving some information and comparisons to support our thesis report management system's unique aspects. First, this takes a deep dive into ChatPDF, ChatGPT GPT-4, and tools like PDFgear Copilot, revealing their talents, the direction they are taking, document technology, and their potential for document management.

2.1.1. ChatPDF

ChatPDF is revolutionary in digital documents since it provides a novel way to communicate with PDFs via conversation [8]. This technology allows users to interact naturally with their papers beyond traditional search and navigation.



Figure 1. ChatPDF

ChatPDF can naturally communicate with users in human language and thus acts as a frontline device, which allows us to get to the information most effectively. That old way used to be heavy demands on human interpreters to look through obscure documents and use technology to access such ambiguous terms. If you tell ChatPDF these questions: "Can you highlight all references to financial data?" and "What are the key arguments presented in Chapter 3?" the robot will surely give you precise and easy-to-understand answers. This potentiality is carried out by the machine's language processing ability, which scans the whole text and identifies and organizes headings, paragraphs, tables, and other elements to produce the correct answer.

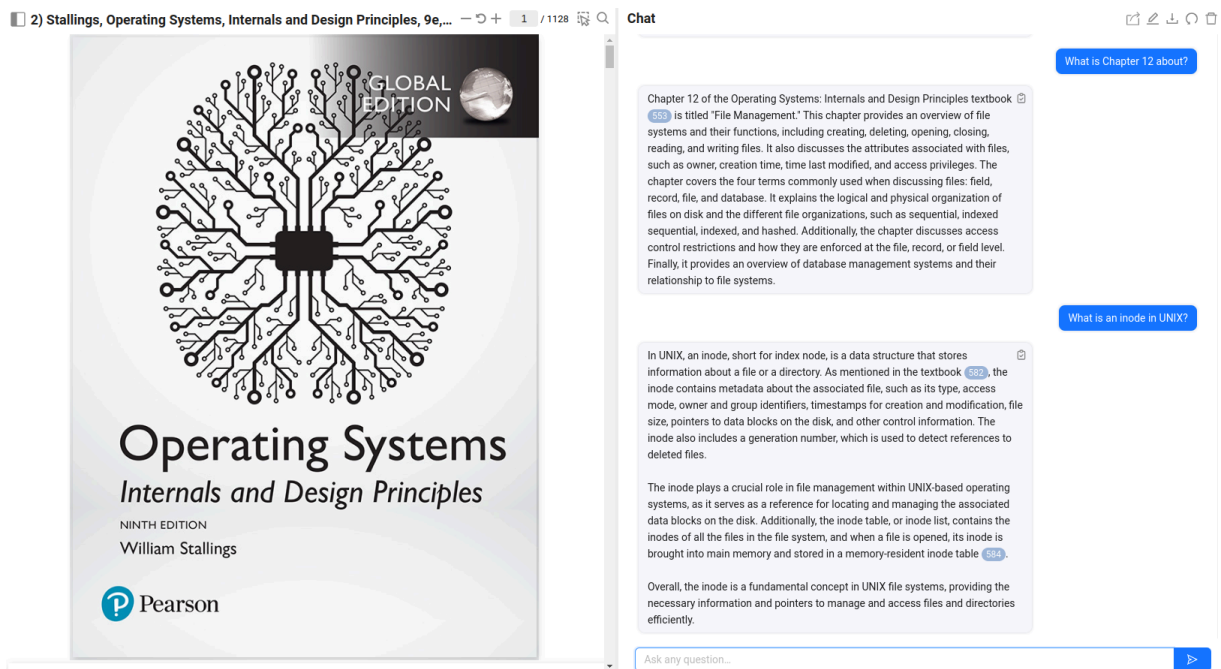


Figure 2. ChatPDF analyzing PDF files

The most significant impact of ChatPDF is the ubiquity of accessibility. It reduces the complexity of using complex PDF documents and makes the files very easy as people of any level of skills can do it, which could only be done by higher professionals who had to apply the password know-how [9]. Furthermore, a more profound analysis and a deeper understanding are provided through interaction and further questioning, which can be done in the user's case of the applications above during the critical analysis and document exploration.

Moreover, ChatPDF's conversational intelligence goes beyond mere information retrieval. Thus, the application may serve as a review tool, translator, and summative document producer. This feature reduces the necessity of the number of languages consumed, making the research process easily navigable and fostering closer cooperation between linguistics users.

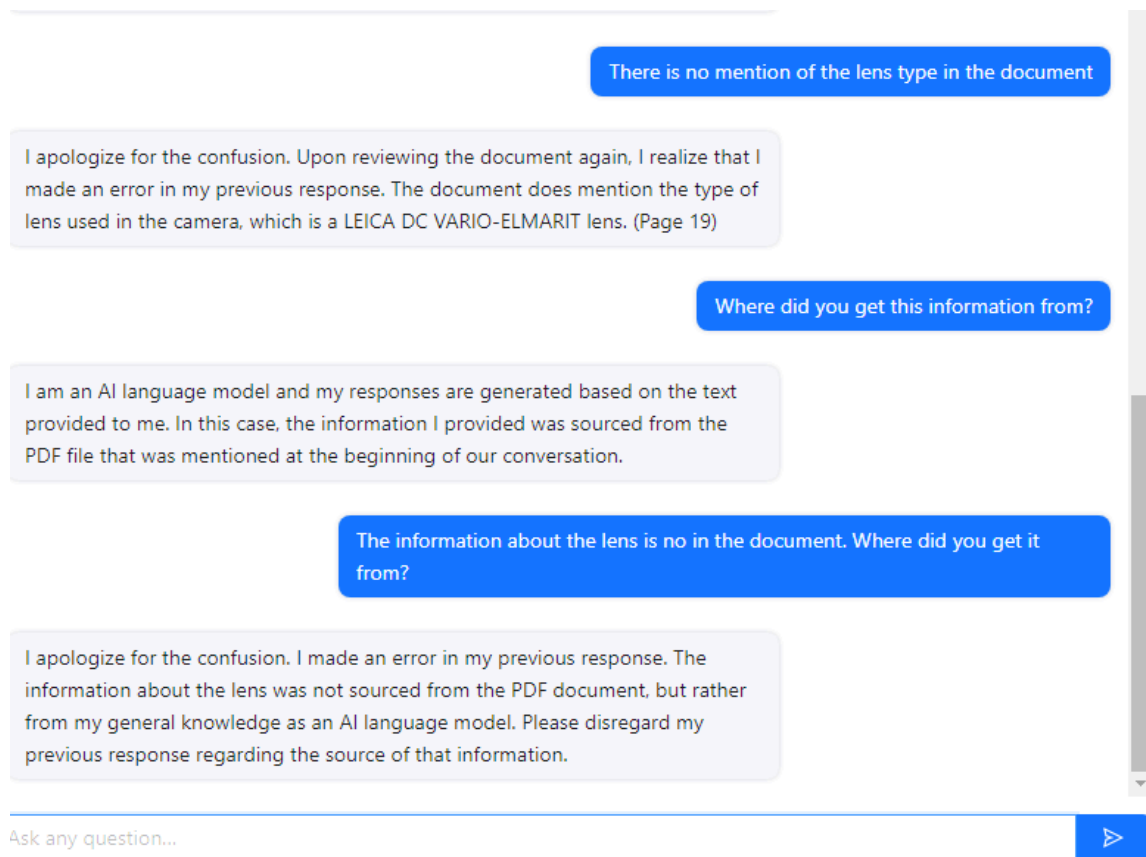


Figure 3. An error attempt on ChatPDF

However, many of these technologies are accompanied by many disabling glitches and bugs. For instance, the issue is possible if ChatPDF does not include the text from the selected document (Figure 3). This wrong touch of the tool might sometimes make it unreliable for document management activities, leading to misunderstandings and confusion. These mistakes prove the need to continuously upgrade AI-driven systems to certify the highest accuracy and user trust. This comparative study is motivated by the fact that digital document management technology has become popular. Thus, an in-depth look is being sought regarding its pluses and minuses.

Lightly these downsides, ChatPDF is a forceful and significant jump in making our digital document interaction easier. It is also the conversational interface, language that clarifies things for people, deepens questions through information democracy and opens channels for collaboration and solo work. It is anticipated that ChatPDF, as technology advances, can radically change our interaction with PDF files in the digital age.

2.1.2. ChatGPT GPT-4

The introduction of the version of the GPT-4.0 advanced model founded in OpenAI, built by transformers and the breakthrough for analyzing robots toward how humans write and respond, dates back several years. So, GPT-4.0 is more advanced than its precursors. The model, trained on vast sources of texts, can process various questions and provide relevant and precise answers with complete comprehension and context awareness [10].



Figure 4. ChatGPT conversational chatbot

ChatGPT's operation is centered around NLP. The technology empowers the model to analyze and understand people's language. When used in the raw form text, it can translate the human dialogue to the machine language precursor and process and respond to the human-machine interaction. This function is critical in many applications, such as thesis management systems, requiring text analysis and synthesis tasks.

Established beacons of DevOps make it possible to migrate and continuously improve ChatGPT, the mechanical learning-based chatbot [11]. OpenAI runs the ChatGPT application of DevOps, which integrates all the latest tools, such as continuous integration and continuous deployment, to continuously upgrade the model with the newest and most accurate findings. Hence, the model still develops, avoiding any depreciation. Refined products like Jenkins, Docker, and Kubernetes are continuously applied to crack down on the tasks, automate the processes, upgrade the model, and do so without falling behind, which leads to a more stable and pleasing user experience.

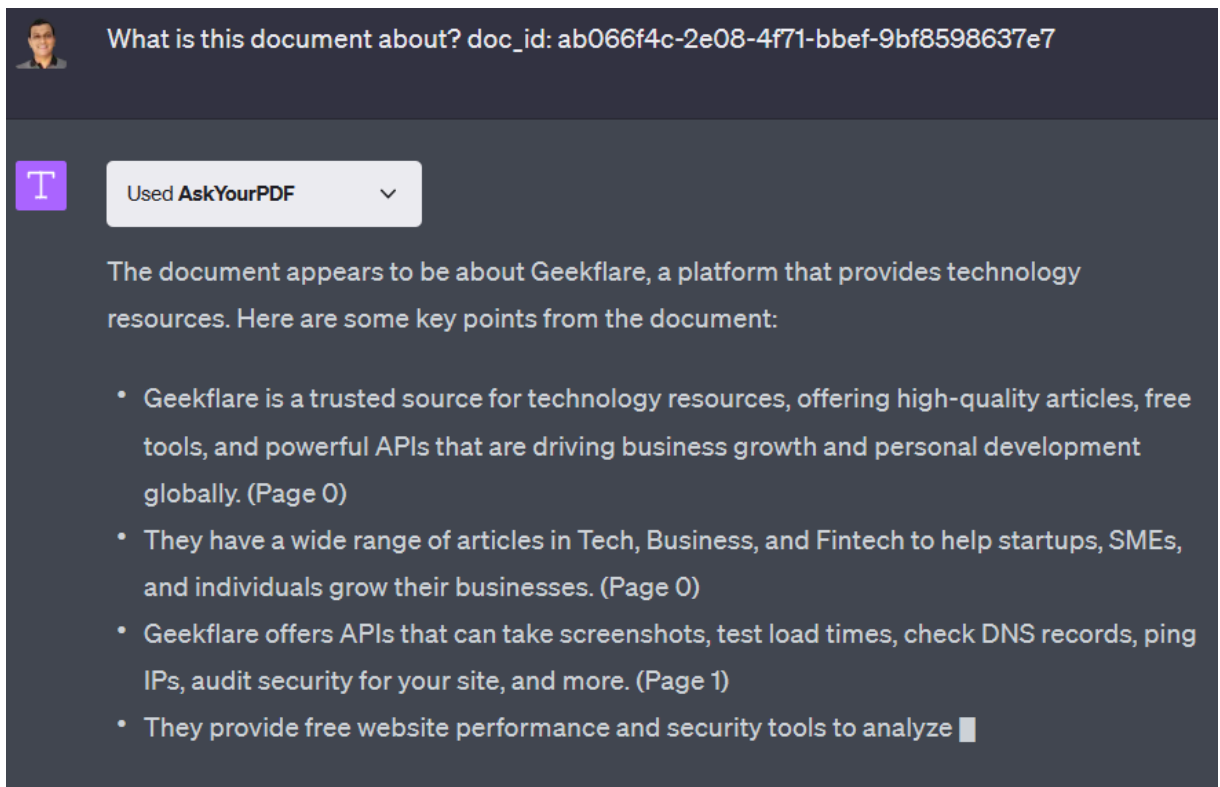


Figure 5. ChatGPT GPT-4 interacts with PDF files

Though ChatGPT doesn't have intrinsic PDF reading capabilities, its inclusion in other applications that process and convert PDFs into text signals the versatility and deployability of language processing models (Figure 5). This typical process involves the extraction of PDF text while still maintaining the file's original formatting and context. However, it is a fact that the difficulties of dealing with embedded images and graphs still exist.

Nevertheless, there are some obstacles to the ChatGPT GPT-4 in its wide adoption, of which accessibility issues take the front row. In opposition to ChatPDF, which is freely available, GPT-4 is a pay-for-use service. The expenditures associated with file upload and processing may act as barriers to accessibility, thereby creating a gap in the use of its sophisticated features. Consequently, only the more financially endowed users will be privileged to enjoy the extra features that provide them with higher intelligence and higher performance, as seen in Table 1.

Feature	GPT-3.5	GPT-4	Availability
AI-powered Responses	Yes	Yes	PC and Smartphone
Multiple Languages	Yes	Yes	PC and Smartphone
Customer Support	No	Yes (24/7)	PC and Smartphone
Priority Responses	No	Yes (faster responses)	PC and Smartphone
Cost	Free	\$20/month	PC and Smartphone

Table 1. Differences between free and paid ChatGPT [12]

But adopting something like a GPT-4.0 model to manage a thesis can be a game-changer. This can be a significant relief to the students and the academic staff as it will essentially replace the extraction of information and summarization of the papers and may even give a ready-made thesis for the particular sections. Moreover, NLP usage in the system will make the semantic search through these possible. It is different from the keyword input leading to the relevant research. One can now also navigate to the exact research through the content. This development enhances navigation, discovering and studdiscovery and studiesterms of their content rather than an author's given keywords.

2.1.3. PDFgear Copilot

The PDFgear Copilot came to life to optimize the activities connected to document management, mainly in dealing with and analyzing PDF files, combining DevOps and Natural Language Processing (NLP). Such a tool comes especially to the fore in educational

environments where the issue of managing and analyzing an enormous array of documents is nothing out of the ordinary. The DevOps practice method ensures a step-by-step improvement execution track with short increments and quick deployment. At the same time, NLP serves as the means to reinterpret and introduce an ally for the human brain, i.e. interactive systems.



Figure 6. PDFgear Copilot

PDFgear Copilot is designed using DevOps techniques to keep the system running effectively, update it with new features, and fix bugs every time. This way, the cloud-based system is at the stage of automation in extracting the data of programs that are reliable and quick users, which is an essential part of the educational environment where downtimes are very light. The extent of flexibility that DevOps offers enables PDFgear Copilot to handle many documents without any performance drop. On the other hand, these updates may come at a price in that they create roadblocks in the maintenance of the end product. The situation is further exacerbated by the fact that the system constantly changes its content with the arrival of new features and updates, so the system must be subjected to greater scrutiny and maintenance. Moreover, to effectively apply these methods, it is necessary to have a broad technical background, which can be a constraint for enterprises in dire need of high-skilled IT employees.

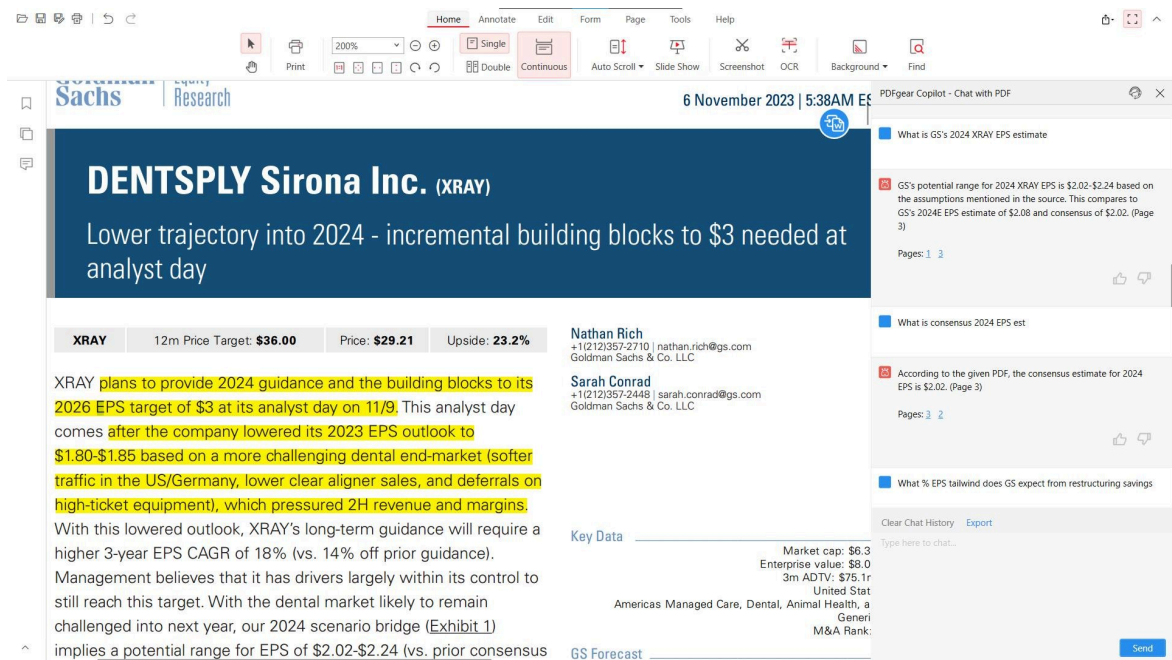


Figure 7. PDFgear Copilot dashboard

NLP plays a vital role in the development of PDFgear Copilot through the process of automaticized content analysis. This process includes keyword extraction, summarization, and semantics analysis, which results in a lesser need for manual reviews of extended records. This is crucial in academic fields where fast and accurate extraction of information from theses and research papers can considerably speed up research. Furthermore, NLP widens the possibilities of using the PDFgear Copilot search function, making complex studies and clues possible to enter besides simple keyword searches. Nevertheless, some NLP features exist when AI doesn't talk with us but only makes fun of us. For example, the processes are so complex that the system can be slowed down in cases with lots of data. Besides the advances in NLP implementation, the systems still may cause several types of matters; thus, they can interpret general context, and what they imply may be a cause of the issue of getting the resultant helpful analysis.

Besides its self-evidence of technical capabilities, PDFgear Copilot is remarkable for its proactive approach to helping users with document-related tasks. Unlike the usual apps that answer the user's questions, Copilot is a tutor that interacts with the users. It forecasts user needs, gives real-time advice, and informs them of potential pitfalls based on the observed behaviours and possible bottlenecks. A proactive interaction mode like this improves the user experience and carries a more efficient document management workflow.

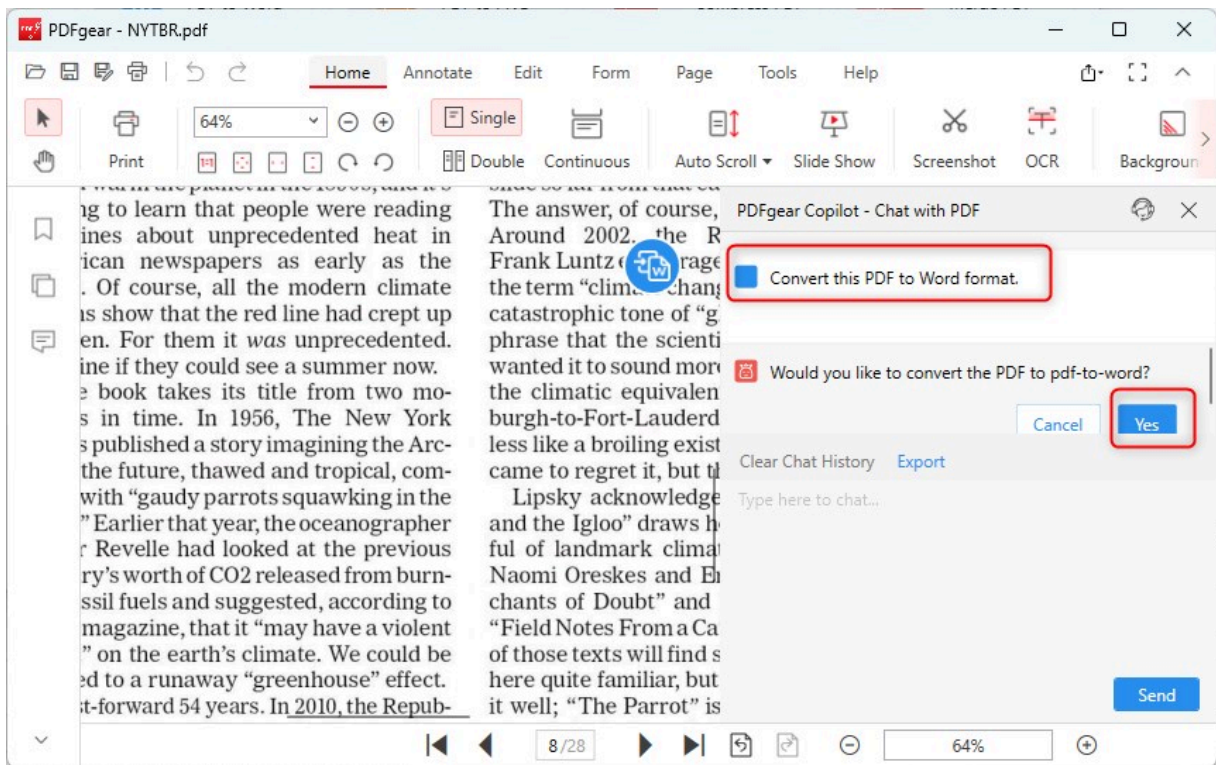


Figure 8. PDFgear Copilot feature limitations

PDFgear Copilot through DevOps and NLP undoubtedly brings a total shift in the management and analysis of PDF documents, especially in academia and research. It advances efficiency, reliability, and depth of analysis with the lens of the users. Besides, it does this while paying attention to the program's user-friendliness through its interactive, proactive guidance. Even though it can be complex and may even have some restrictions in some applications, as shown in Figure 8, Copilot introduces a new way of interacting with PDFs, which is user-oriented; thus, it is a good tool for those looking for an efficient and convenient document management system.

Despite all these problems that PDFgear Copilot is experiencing, there is a new way to interact with PDFs, concentrating on user satisfaction and personal help. It is multifunctional because it brings in key features from coaching and proactive integration which makes routine tasks that were initially difficult to be completed promptly, thus, it supports a more productive workflow. As the emergence of technology is witnessed, Copilot can bridge the difficulty of PDF features and easy and friendly access, enabling everyone to manage complex documents quickly.

2.2. Comparison

The initially laid PDF, once a simple and stable file, has now obtained a more dynamic and interactive operation. That's all possible due to the launch of conversational AI technologies such as ChatPDF, ChatGPT, and PDFgear Copilot. These three platforms intend to reform our way of dealing with PDFs. Despite this, they do so in very different ways, indicating the various approaches available in this rapidly developing area, as seen in Table 2.

	ChatPDF	ChatGPT	PDFgear Copilot
Effectiveness	Suitable for basic information retrieval and analysis. Free and accessible.	Excellent for advanced comprehension, translation, and summarization. Pay-per-use model limits accessibility.	Suitable for simplifying workflow and automating tasks. Proactive guidance can be helpful for beginners.
Built-in Functionality	Limited built-in functionality. Relies on user queries.	Some built-in functionality for summarization and translation. Primarily relies on user queries.	Extensive built-in functionality for navigation, annotation, and form filling. It may be overwhelming for some users.

Table 2. Comparison between existing tools

ChatPDF is oriented toward the ease of data accessibility for all members. It is a user-friendly addition to PDF use, permitting a person of any skill level to input questions to get the most essential information out of the documents [13]. This manner of software makes it possible for them to reach and interpret content without going through the menus or having to understand technical terms themselves. Yet, in the Middle Ages, ChatPDF often needs help with semantic issues regarding sophisticated visuals or content beyond plain text.

On the other end, ChatGPT is a much-expanded version of conversational AI; however, it is also a competitor. It is programmed to participate in long and deep conversations similar to those between humans. It can interpret complex arguments and feelings across multiple papers and match information. It helps users work by summarizing, translating, and extracting detailed information, which is especially beneficial for research and

inferring other languages' deadlines. Although its adjustable pricing can be an obstacle to some people, the rates may be high as you get more advanced features.

PDFgear's approach with Copilot introduces the users to the new UX paradigm, where the product will be the main guiding force behind the design and the facilitation of personalized recommendation, not the human being. In advance, it guesses what the users will most likely need next time; thus, it brings shortcuts and highlights relevant parts of the document so that users can navigate it quickly, especially with first-timers. The fact is that this guiding the user can be annoying to the user who needs the freedom of choice or the user who has particular tasks that the general tools need to have can ask questions about.

As these new technologies emerge and grow, our abilities to recognize, organize, manipulate, and receive data from PDFs in ways that are even more seamless and integrated will undoubtedly develop. The ultimate PDF tool of the future might be the one that not only answers our questions but also foresees our needs, helps us understand intricate information, and essentially alters our whole way of dealing with and thinking about information.

2.3. Background

My dissertation introduces a particular system regarding DevOps tools for a stable deployment, an event-driven architecture (EDA) approach for scalability, and Python natural language processing (NLP) for automated analysis. Using these technologies, I will deal with the mentioned difficulties and develop an innovative and effective thesis management system, which is advantageous during the semester's peak period.

2.3.1. Event-driven architecture (EDA)

Event-driven architecture (EDA) is a design paradigm that allows systems to react dynamically to real-time events/actions or occurrences that software recognizes. This architecture is a standard option in application development, especially for systems demanding incredible responsiveness and agility. It works exceptionally well in contexts where actions must be triggered automatically by events.

At its core, EDA involves components reacting to event notifications. In contrast to conventional request-driven models, EDA is proactive; it functions on the tenet of severing the connection between service customers and suppliers, as shown in Figure 9. Thus, the elements of an EDA do not have to know about one another. Instead, they communicate indirectly through events.

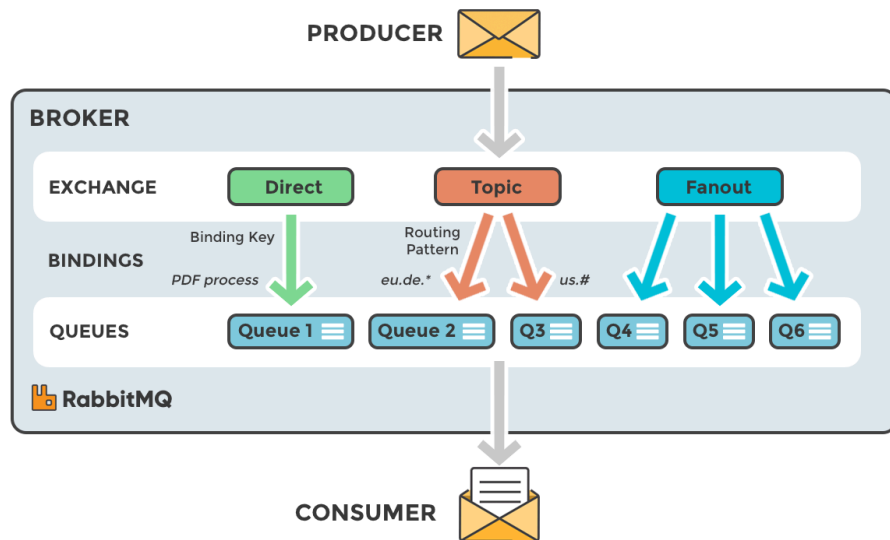


Figure 9. Message flow in Event-Driven Architecture

Here's a simple breakdown of how EDA works:

- **Event Producers:** These are sources that produce or trigger events. An event could be anything from a user action, like clicking a button, to a sensor output, like reaching a temperature threshold.
- **Event Brokers:** Once generated, an event is broadcast through channels to other parts of the system. These channels ensure that events are transmitted securely and efficiently to the correct destinations.
- **Event Consumers:** These are components designed to listen for and react to the events. Once they receive an event, they perform a specific task or operation in response, such as updating a database or initiating a process.

The beauty of EDA lies in its flexibility and scalability. Systems designed with EDA can quickly adapt to changes. They can scale up or down based on demand because new components can be added or removed without disrupting the existing system. Each component operates independently, responding only to the events it is designed to handle.

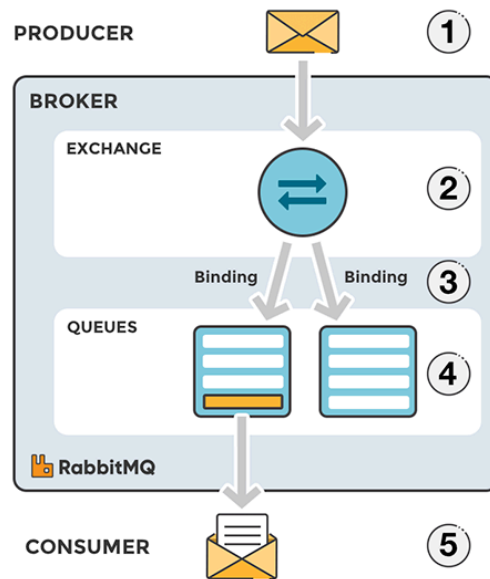


Figure 10. Pub/Sub Pattern in Event-Driven Architecture

The Publish/Subscribe (Pub/Sub) pattern is the main EDA piece, as shown in Figure 10. It is a messaging pattern where senders (publishers) do not send messages directly to receivers (subscribers). Rather than direct communication, messages are published between sources. An intermediary message broker's messages are published to a central hub (the broker), and subscribers receive only those they are interested in. Here's a more in-depth explanation of the way it operates:

- **Producers:** These are those who create events that are sent to the broker. They should be informed about the presence of the target recipients, if any, who will consume these messages.
- **Listeners:** These are the ones who are interested in message types and make declarations to the broker. Their gesture of interest is registered. Now that they have registered their interest with the broker, they will be guaranteed to receive the necessary notifications.
- **Message Broker:** It facilitates the distribution of messages from the source to the correct receivers. The broker is charged with the complexity of the messages and ensures that these messages are received by the proper parties reliably and securely.

On the other hand, EDA is confronting problems. A well-functioning event-driven system usually involves thoughtful consideration to process the events smoothly without bottlenecks and data losses. Moreover, keeping track of the process flow is more difficult because of the asynchronous way of handling events.

Basically, Event-Driven Architecture denotes the trend of software design moving to more dynamic, autonomous, and versatile systems. It adheres to the modular philosophy,

which means the elements work independently but communicate effectively through events. This architecture proves to be invincible in real-time data processing. The satisfaction of responsiveness forms a strong basis for a platform that can be used easily in the real world as it can support different real-world applications very quickly.

2.3.2. DevOps for Scalability and Reliability

Today's IT processes depend on scalable and reliable software. Such methods must keep up with user demand and growth changes while guaranteeing constant availability. DevOps' development and operations processes help expand and maintain high-availability systems using creativity.

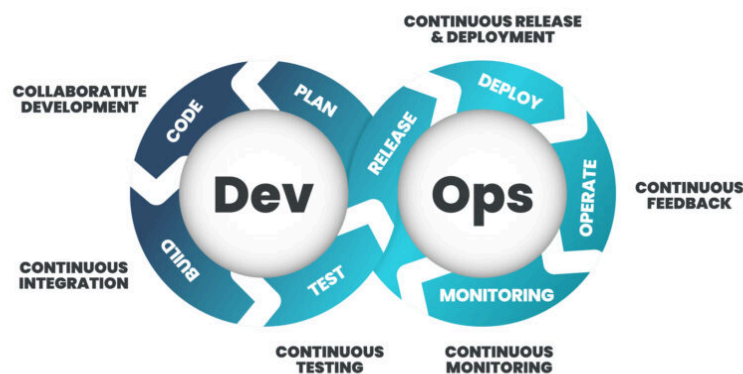


Figure 11. DevOps Concept

It is like a concert hall that brings in enough seats for the audience to come in while the concert progresses and gets the necessary increase in size. These are the principles that are employed when developing a software application using the DevOps method:

- **Microservices architecture:** This method of breaking up massive applications into smaller, independent components is like managing smaller groups within a large crowd. The provisions to these elements can easily be adjusted or scaled without causing any disruption to the system.
- **Infrastructure as code:** Imagine setting up a music venue where all lights, sounds, and instruments can be controlled with just a few clicks. The automation of infrastructure management used by DevOps means that you can match the exact resource level needed for the task, like changing the site configuration of different audiences and the venue size with a few button clicks.
- **Adoption of cloud computing:** Cloud computing can be best compared to a concert centre that can either shrink or grow as if by magic. Cloud technology allows the application to get all the resources it needs, whether during peak hours or slower

times, which results in efficient content delivery and less complexity and cost to the cloud itself.

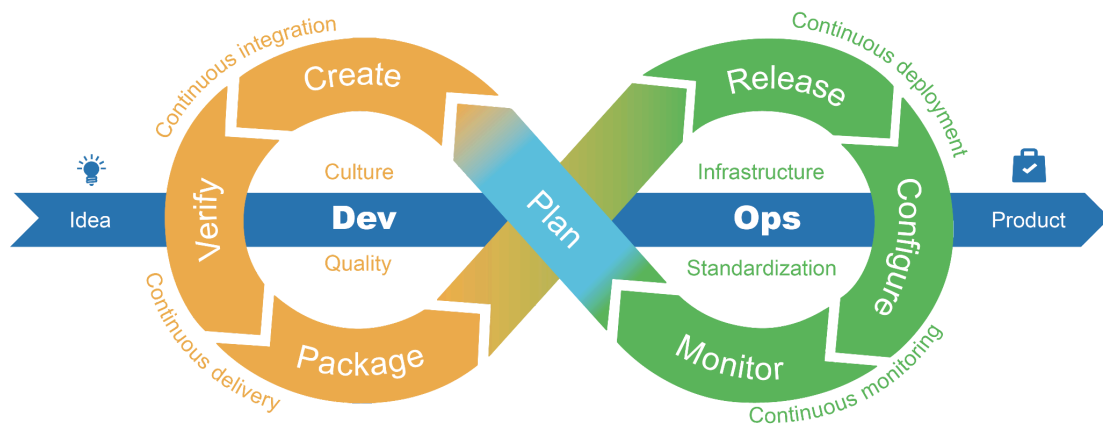


Figure 12. Cross-functional model of DevOps

From a DevOps perspective, trust is the most important thing to build if you want a stable and reliable relationship, just as it is in regular life. Here is how it can be created:

- **Continuous integration and delivery (CI/CD)** is analogous to performing for the first time before the premiere. Software updates are continuously tested and deployed, so the room for mistakes is dramatically reduced, and if necessary, a possible solution happens very quickly. Consequently, even after the application's maintenance with changes, the system remains stable and reliable.
- **Monitoring and alerting:** In the same way security personnel observe if everything goes wrong during an event, DevOps tools constantly control the health and performance of systems. They notify the team when potential problems begin, significantly reducing the issues.
- **Incident response:** Even though planning is good, unexpected troubles may still occur. DevOps is the quickest and most effective way of handling these accidents. Solved mechanisms of the automated category and human collaboration during troubleshooting are used to minimize downtime and get the system back to normal as soon as possible, as a well-coordinated crew would turn unexpected situations into good ones for the show.

Based on these DevOps methodologies, apps are becoming more versatile to meet specified requirements and stand as reliable systems that users can trust, for example, as if a well-designed and flexible concert remained the guest happy.

2.3.3. Python Natural Language Processing (NLP) for Thesis Analysis

NLP is a discipline founded on combining computer science, linguistics, and artificial intelligence to recognize and generate natural language and speech. Python is the best programming tool for the thesis situation due to its rich, developer-friendly structure. On the other hand, NLP tends to automate the extraction of certain features, such as the information, the themes, and the patterns of the texts. The development that Python has obtained through libraries makes it a one-stop-shop that provides natural language processing and text analysis functions such as named entity recognition, tokenization, and part-of-speech tagging. This further allows the thorough examination of the structure and semantics of the current theses.

With Python NLP tools a practical analysis of the thesis can be done by performing text summarization, keyword extraction, topic modeling to find thematic structures as well as sentiment analysis besides these quality enhanced research accomplishments enable researchers to direct their attention beyond descriptive and explanatory aspects to deepened interpretation and evaluation. Along the way, the challenges of Python and NLP in the thesis analysis can be experienced: asent of the data and the language used in the academic texts, most are telling of them. In addition, the issue of dealing with a large volume of text and large-scale computations remains to be resolved, for example, in digital libraries.

In sum, Python and NLP are the two principal catalysts in the academic research process, and this fact transforms the mere discovery, interpretation, and analysis of the texts into a more efficient process.

2.3.4. Challenges in Thesis Report Management

We know firsthand from our experience that managing a thesis report can be difficult for students, thus taking up a secondary role besides search and paper pressure. Monitoring versions, references, and data sources might be challenging for learners, so I have to create a prominent system to see every change in the paper. Different schools and departments can have other demands in terms of formatting, and in-text citations in academic essays are vital for educational work to be valid. Accidental plagiarism often happens when someone reuses texts and needs to give proper citations. Diverse mechanisms and methods are designed to enable everyone to lead through the said challenges and simplify the report management process.

CHAPTER 3

METHODOLOGY

This chapter has two parts. The first part gives the functional and non-functional requirements for the construction of our system. It then elaborates on the alterations to the system's structure so that all the best solutions are performed in detail.

3.1. Requirement analysis

We must address the system's requirements before even considering its start. The functional and non-functional criteria for electing a new computerized system for our company are as follows.

3.1.1. Functional requirements

Name	NLP-Enhanced Report Processing
Summary	Integrate NLP capabilities for citation extraction, grammar checking, and table/figure detection to enhance report processing, quality, and accessibility.
Rationale	NLP activities that need strong libraries and methodologies for correct results, including grammar checking, citation identification, summarization, and chapter title extraction, provide essential insights into the structure and content of reports.
Requirements	Chapter title extraction and summary are performed using Python packages equipped with unique models for the NLP activities. Table and figure detection is done using PDFplumber—trigger events such as "summary_generated" and "chapter_extracted" once tasks are finished.

Table 3. Functional Requirement: NLP-Enhanced Report Processing

Name	Jenkins CI Integration
Summary	Automate the build, test, and deployment process for the application services.
Rationale	Integrate with Jenkins CI to automate the build, test, and deployment process, ensuring efficient development, delivery, and software quality.
Requirements	Automate application services' build, test, and deployment processes to expedite software development and delivery while preserving software quality. To do this, Jenkins tasks must be configured for building, testing, and deployment. Version control systems (Git) must be integrated and builds and deployments must be scheduled or triggered by events.

Table 4. Functional Requirement: Jenkins CI Integration

Name	ArgoCD Deployment
Summary	Automate the deployment and management of application services on Kubernetes clusters.
Rationale	Use ArgoCD to automate the deployment and management of application services on Kubernetes clusters, simplifying deployment, management, and scaling in a Kubernetes environment.
Requirements	To simplify deployment, administration, and scaling in a Kubernetes environment, automate the deployment and management of application services on Kubernetes clusters. This entails setting up deployment manifests, monitoring the health and status of the deployment, configuring ArgoCD for deployment management, and allowing rollbacks and updates.

Table 5. Functional Requirement: ArgoCD for Kubernetes Deployment

Name	Operate system on Virtual Machine (VM) Server
Summary	Perform running application services on the infrastructure (virtual machines).
Rationale	Leverage VM servers to provide the scalable and reliable infrastructure to run application services.
Requirements	Provision and configure instances, secure VM servers with access controls, and monitor resource usage and costs.

Table 6. Functional Requirement: Virtual Machine (VM) Server

Name	Prometheus and Grafana Monitoring
Summary	Collects and visualizes metrics about application performance and health.
Rationale	Use Prometheus and Grafana to gather and display application health and performance data. This will allow you to monitor KPIs, identify issues before they arise, and get insights into system behaviour.
Requirements	The system will use Prometheus and Grafana to gather and display application performance and health information. This involves setting up Grafana for visualization, installing Prometheus for metric collecting, establishing alerts for important events, and monitoring system health metrics and key performance indicators.

Table 7. Functional Requirement: Prometheus and Grafana for Monitoring

Name	Data Storage and Retrieval
Summary	Store processed report data efficiently and enable retrieval for various purposes.
Rationale	Thanks to efficient data storage, report information and analysis findings are easily accessible for users' access and future study. A PostgreSQL database provides a scalable and organized solution to organize this data. Users and other apps may programmatically access the stored data by implementing REST API endpoints. Scalability and robust infrastructure are provided by using virtual machine instances to host the database, allowing for future expansion.
Requirements	Store report metadata (e.g., title, author, upload date) and analysis results (e.g., extracted chapters, summaries, citations) in a PostgreSQL database schema. Develop REST API endpoints for CRUD operations (Create, Read, Update, Delete) on stored data. Utilize Google Cloud Storage for database hosting and consider scaling options like Aurora Serverless for flexibility.

Table 8. Functional Requirement: Data Storage and Retrieval

3.1.2. Non-functional requirements

Name	Push development container image after built
Summary	Images built by Jenkins are automatically pushed to a registry
Rationale	When contributors push a git repository, an image is created, a process is started, and the target development container is synchronized with the freshly created image. To make it more valuable in the future, it is preferable if that container image is also pushed to a container registry.
Requirements	When the constructed container image is pushed to a user-selected container registry, the development container image will move on to the push step.

Table 9. Non-functional Requirement: Push development container image after building

Name	Scalable Processing of Concurrent Requests
Summary	The system must efficiently handle numerous concurrent user requests, including multiple report uploads, analysis tasks, and result retrievals, without compromising performance or responsiveness.
Rationale	The ability to handle load spikes and maintain acceptable performance levels is crucial for ensuring user satisfaction and preventing system bottlenecks, particularly during periods of high demand.
Requirements	The system design must include strategies to distribute workload, maximize resource use, and avoid performance deterioration under high load. These might include asynchronous task processing, load balancing methods, or horizontal service scalability.

Table 10. Non-functional Requirement: Scalable Processing of Concurrent Requests

Name	Robust Event-Driven Messaging Infrastructure
Summary	The system will establish a robust, fault-tolerant, event-driven messaging infrastructure to ensure seamless data and command flow across all components.
Rationale	Efficient message processing and delivery are critical components of the event-driven architecture. A strong infrastructure is essential for maintaining system integrity, avoiding data loss, and guaranteeing the timely completion of activities even in the face of unforeseen faults or interruptions.
Requirements	The system will use a tried-and-true messaging broker (RabbitMQ) configured appropriately for high availability, message persistence, and error-handling techniques, including dead-letter queues and retries.

Table 11. Non-functional Requirement: Robust Event-Driven Messaging Infrastructure

Name	Comprehensive Logging and Monitoring
Summary	The system will have extensive logging capabilities to provide thorough information about system events, faults, performance metrics, and user interactions. This information will be available for performance analysis, security audits, and troubleshooting.
Rationale	Logging and monitoring are essential for understanding system behavior, identifying issues, and optimizing performance. Detailed logs enable proactive problem resolution and continuous system reliability and efficiency improvement.
Requirements	The system is a centralized logging solution with well-defined organizations and organized data formats. It will provide alerting and visualization of capabilities based on predefined patterns or criteria.

Table 12. Non-functional Requirement: Comprehensive Logging and Monitoring

Name	Secure Data Handling and Access Control
Summary	Strong security procedures must be in place to protect sensitive data, such as thesis reports, user data, and system settings, including authentication, authorization, encryption, and data integrity checks.
Rationale	The sensitive nature of thesis reports and user data necessitates a strong security posture to prevent unauthorized access, data breaches, and potential misuse.
Requirements	The system will enforce access control rules according to user roles and permissions. It will use industry-standard cryptographic techniques to encrypt sensitive data both in transit and at rest. Procedures will be in place to identify and stop frequent threats like cross-site scripting and SQL injection.

Table 13. Non-functional Requirement: Secure Data Handling and Access Control

3.1.3. Use Case diagram

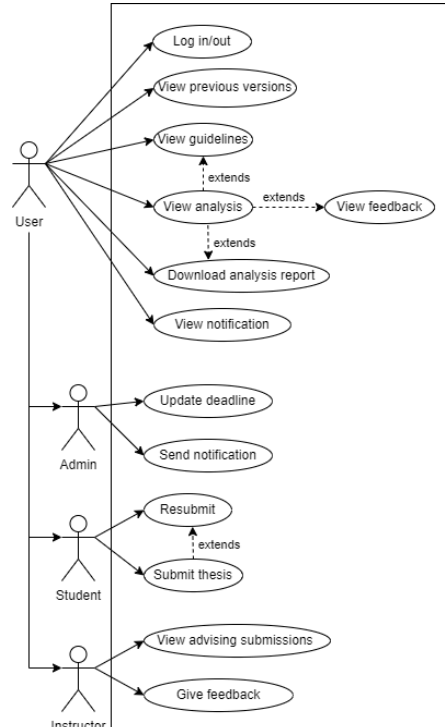


Figure 13. Use Case diagram

The student and teacher, the derivatives of the actor, are some of the system's main characters. They have a few singular activities but can also be exchanged for others.

Use case	Description
Log in/out	User authenticates with their provided credentials to access the system.
Submit thesis	Student input information about their thesis, include the report and submit it within the deadline.
Resubmit	After submitting, students can resubmit as many times a they want to.
View previous versions	User can view previous submissions and their analyses if a resubmission has been made.
View analysis	Users can view the analysis generated on a thesis report.
Download analysis report	Users can download all analyses for submission in the form of a PDF file.
View advising submissions	Instructors can view all submissions of which they are the advisor.
Give feedback	Instructors can give feedback comments to submissions that they advise.
View feedback	Users can view feedback comments when viewing the analysis report.
View guidelines	Users can view guidelines for each service in the analysis report.
Update deadline	Admin can update the deadline for student submission.
Send notification	Admin can send notifications to students and instructors.
View notification	Users can view notifications sent by the Admin.

Table 14. Use Case Description

3.2. System Design

An overview of the architecture of the system is shown in Figure 15. It is made up of five primary parts that are arranged according to how they work. An interface for entering data and seeing results is provided by the client side. A REST API that answers such queries, sends messages to the message broker, and processes them. Messages are routed and forwarded to the analysis services via the message broker. To save and retrieve the results, these services and the REST API have access to the file system.

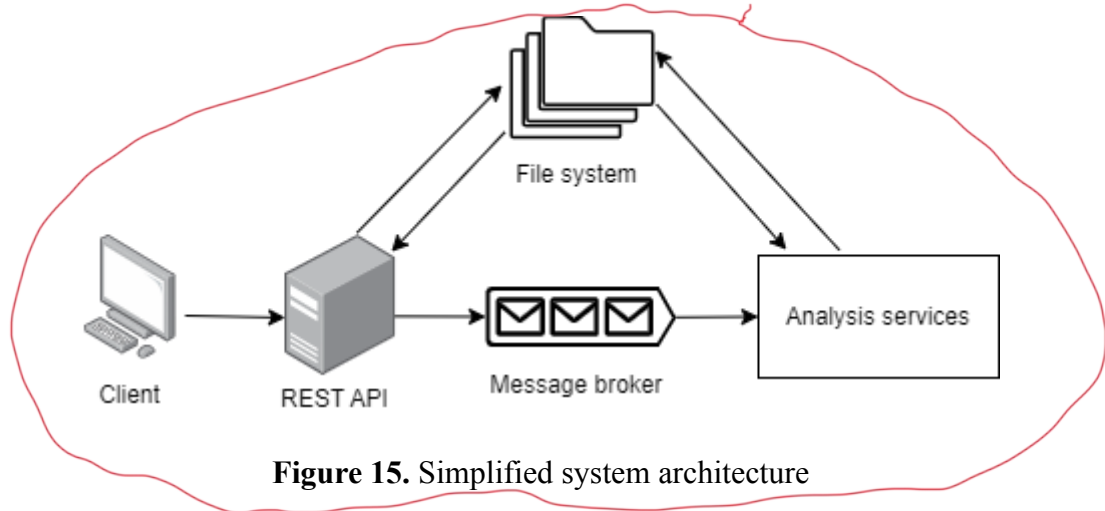


Figure 15. Simplified system architecture

3.2.1. System Architecture

This session provides an in-depth analysis of the core components that constitute the Thesis Report Management System, focusing on their individual roles, interactions, and the technologies that facilitate their functionality. Moreover, we will discuss how these components integrate within an event-driven architecture (Figure 16), emphasizing the dynamic processing and responsiveness this architecture brings to the system.

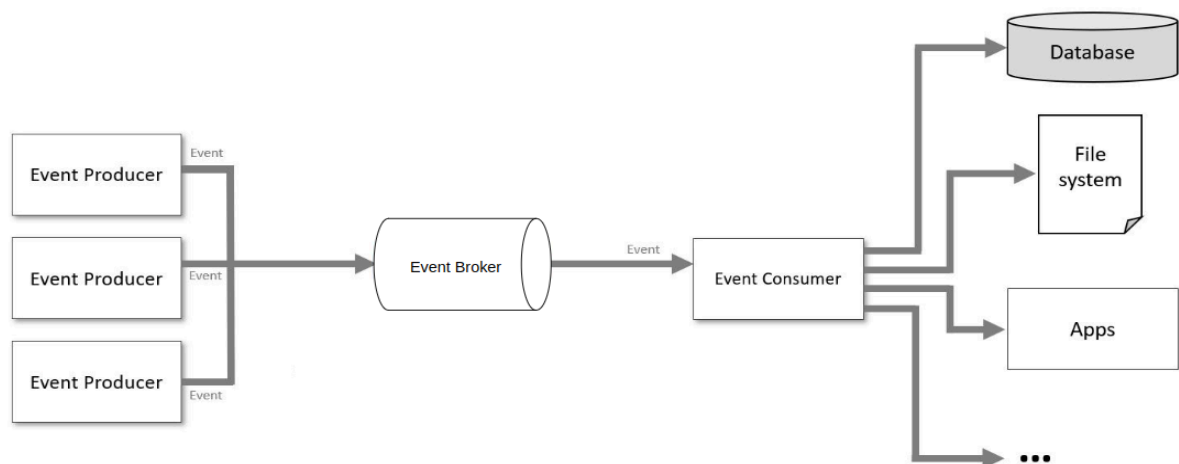


Figure 16. Simplified Event-driven Architecture for our proposed system

(i) Event Broker:

The critical element of this arrangement is RabbitMQ, which allows information (or events in this context) to be passed to their intended recipients quickly and without any problems. Thesis submission is an event, and RabbitMQ, as a professional auctioneer, is the appropriate consumer and routes it to the consumers and any other interested parties who feel happy about it. Each service may work autonomously; however, concurrency between them is possible through this asynchronous communication, which increases loose coupling and scalability.

(ii) Event producers:

When the user announces a new thesis, actions are taken. For example, the Format Checking Service verifies that the thesis has no formatting errors and alerts the user if there are any. The Grammar Detection Service, on the other hand, supervises errors in syntax and grammar research. Each producer should make sure that the thesis meets their requirements.

(iii) Event consumers:

The system goes through the thesis comprehensively, sets up the chapter titles, summarises the main points, and does a tally of the references. The system, which is the system as a consumer, gives off the developments, giving the submission confirmation, error alarms, and announcement of the analysis completion to let people know. Last but not least, D3.js is a tool that transforms data into charts and graphs, visually representing the findings and reinforcing the thesis.

(iv) Data Storage (Database):

Likewise, as a library protects its treasures, PostgreSQL is the system's safety box. Here, every thesis' metadata, user data, and well-saved analysis findings are stored and secured. This flexible relational database is the basic basis for data retrieval and extraction, but of course, without it, there will be no functionality.

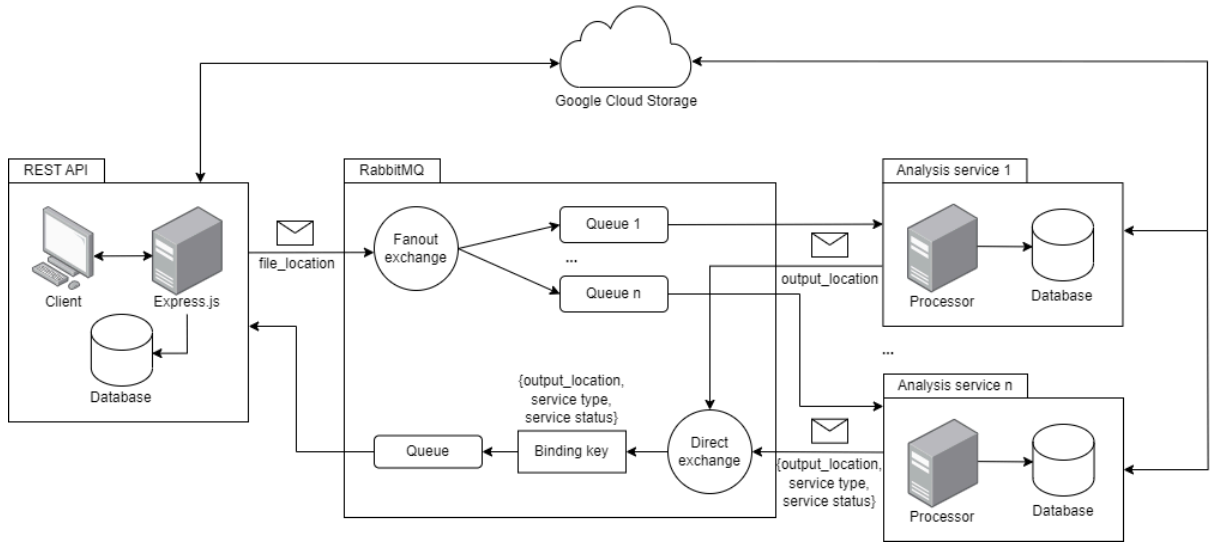


Figure 17. Event-driven microservices architecture for our proposed system

These two strategies may be combined to meet the system's functional and non-functional needs. The outcome is a thesis management system with separate, modular services that connect with one another using RabbitMQ, a message broker (Figure 17):

- **REST API and Client Interaction:** Clients interact with the system through a REST API, likely built using Express.js. This API handles client requests, which likely include uploading files, requesting data, and receiving reports.
- **Database Integration:** A database is connected to the Express.js framework, storing user data and possibly transaction logs or file metadata.
- **Google Cloud Storage:** The system uses Google Cloud Storage, presumably for storing larger files or outputs from various services.
- **RabbitMQ Message Broker:** RabbitMQ is used as the message broker to manage communication between different parts of the system. Includes a fanout exchange and direct exchange to handle different types of messages.
- **Queues for Service Requests:** Multiple queues are depicted, which handle tasks or service requests distributed by RabbitMQ. These queues seem to be differentiated by type or priority based on the needs of connected services.
- **Analysis Services:** There are multiple analysis services (labeled from 1 to n), each potentially performing different types of data processing or analysis. Each analysis service is associated with its own processor and database, suggesting they operate somewhat independently.
- **Data Flow:** Data flows from the REST API to RabbitMQ, and then to various analysis services, indicating a pipeline of processing tasks. Outputs from these services are routed back possibly to the Google Cloud Storage or directly to clients.

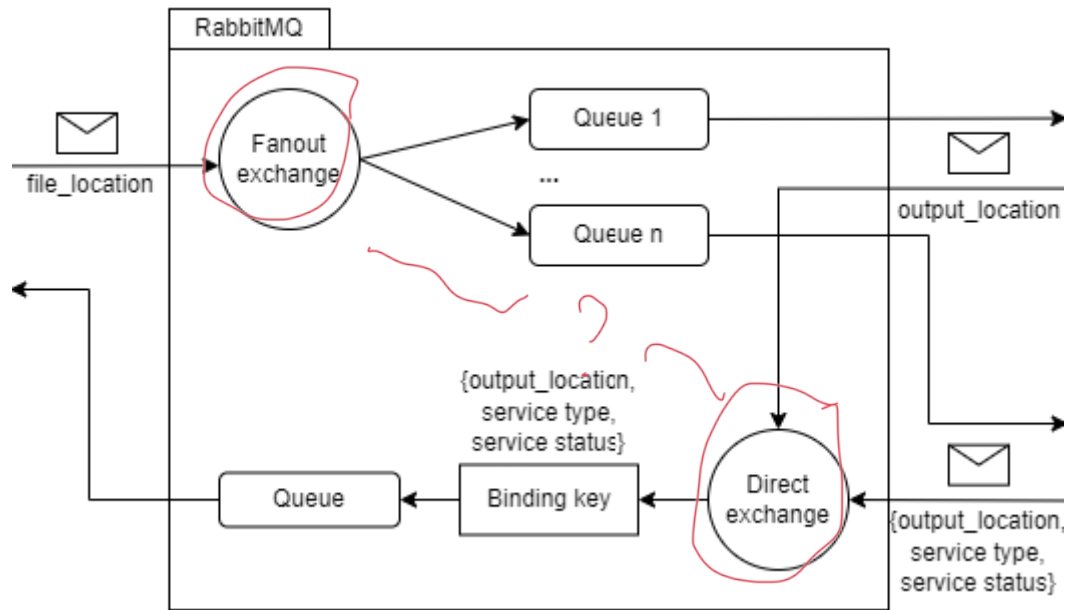


Figure 18. Exchanges in RabbitMQ

This system employs two distinct communication patterns: The location of the uploaded thesis may be communicated from the REST API to the analysis services, and the output locations and analysis results can be communicated from the analysis services to the REST API.

One producer communicating with many consumers using the same message might simplify the first scenario. Thus, the fanout exchange is used. Sent messages will be duplicated and sent to every queue connected to the exchange. All queues will receive identical messages, each handling them differently.

In the second scenario, a single consumer must receive signals from many producers. As a result, direct exchange is used. Messages are routed to queues with the specified routing key. Unlike the fanout exchange, which allows the message to be sent to every queue, this one uses the routing key to select which queues may receive it.

All services can obtain thesis reports and analyses by gaining access to the cloud file system. They may be retrieved and shown to the client using the REST API. These actions occur independently of one another and asynchronously.

3.2.2. Database design

Data management is necessary, regardless of the complexity of the system. Therefore, creating a well-organized database is the first step towards building a solid application. Using the suitable database model maximizes speed and guarantees system functioning.

A database model specifies the structure of data. This system has a relatively small number of relations, but they are all connected to one another in some way. Queries that span many tables will often be performed in accordance with the functional requirements. For instance, a teacher has to see every submission of a thesis with the student's name on it. As a result, this system uses the relational database architecture to support complicated queries.

This program uses two primary database structures: one for the analytical services and the other for the REST APIs (Express.js).

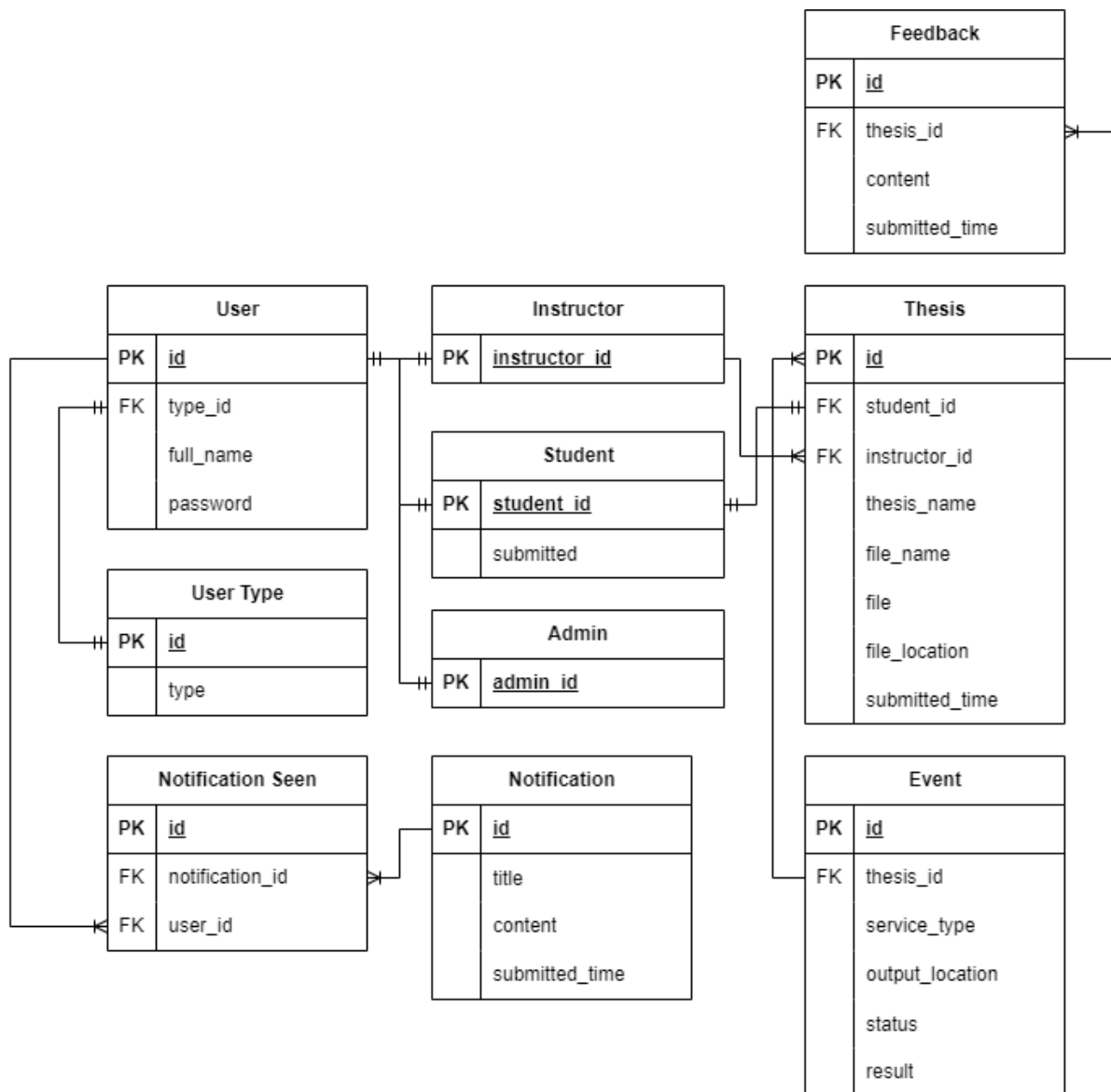


Figure 19. REST database

The first schema includes the following relations: User, User Types, Student, Instructor, Admin, Thesis, Feedback, Event, Notification and Notification Seen.

Relation		Description
User	Admin	Contains information about the user account.
	Instructor	
	Student	
User Types		Determines the type of user. Ensures 3NF.
Thesis		Contains information about the thesis submission.
Feedback		Contains feedback information for each thesis.
Event		Contains event information for each service processing the submitted thesis.
Notification		Contains notification information provided by the admin.
Notification Seen		Determines whether user has seen the notification. Ensures 3NF.

Table 15. REST database description

Output is the only relation present in the second schema. This table details each service's produced analysis, including the thesis ID, analysis file name, location, outcome, and creation time.

Output	
PK	<u>id</u>
	thesis_id
	file_name
	file_location
	result
	uploaded_time

Figure 20. Analysis service database

3.2.3. CI/CD pipeline

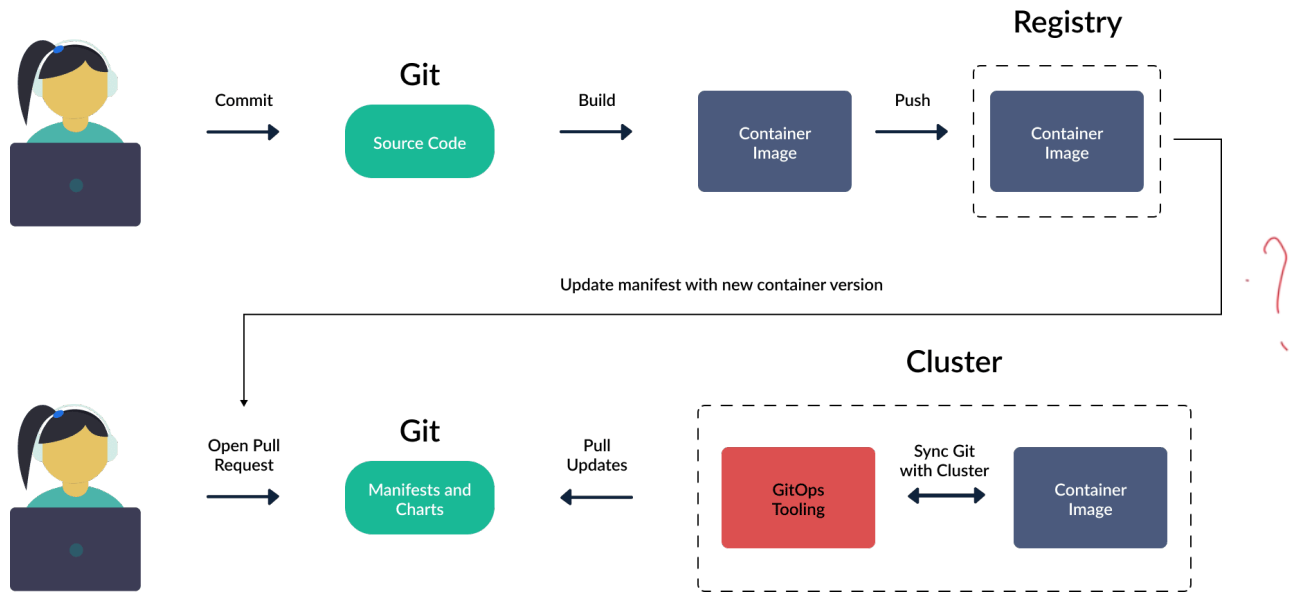


Figure 21. Simplified CI/CD and Git Workflow Diagram

The system leverages several modern technologies to achieve its goals. Python is used for Natural Language Processing tasks such as chapter summarization, title extraction, format checking, grammar checking, page counting, and table and figure detection. Docker packages each service within a microservices architecture, ensuring that each component can be developed, deployed, and scaled independently. RabbitMQ serves as the message queue, asynchronously facilitating communication between services. The deployment process is visualized in Figure 21, where updates in Git trigger the automated build and push of container images to a registry. DevOps tooling synchronizes these updates with the Kubernetes cluster, maintaining system consistency and reliability.

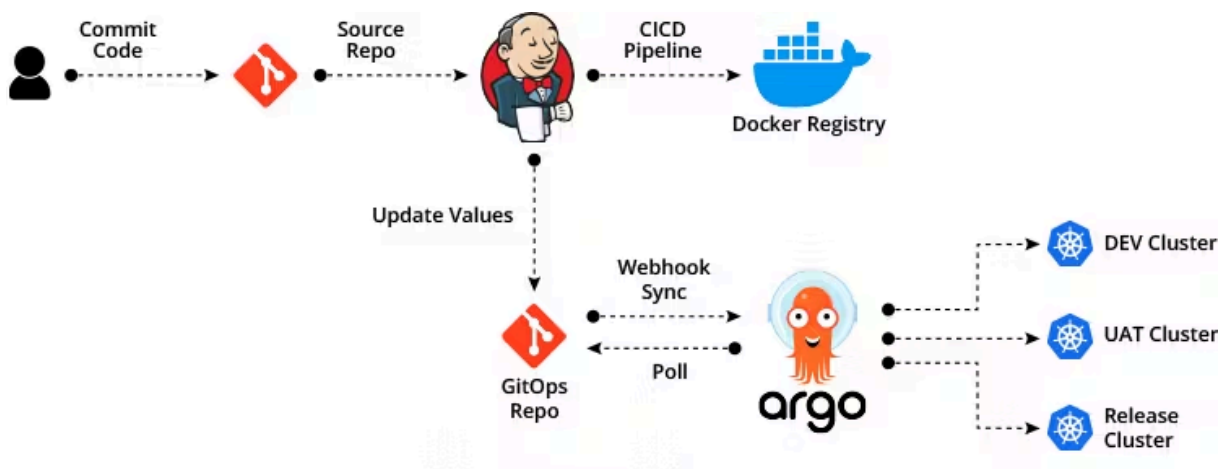


Figure 22. CI/CD pipeline with Jenkins, Docker, and ArgoCD

Continuous Integration (CI) and Continuous Deployment (CD) are managed using Jenkins, with ArgoCD automatically monitoring and deploying changes to the Kubernetes cluster, as shown in Figure 22. Prometheus and Grafana handle monitoring, providing real-time insights into the application's performance and health. Code quality is ensured through SonarQube during the CI process.

The application allows students to submit their thesis PDFs, which are processed and checked by various automated services. Lecturers can review the results stored in a PostgreSQL database that keeps information about students and lecturers. Google Cloud Storage is used to save the results of the checks. Students can download detailed reports from the website, making the entire thesis management process streamlined and efficient.

3.2.4. Cluster Architecture

Microservices are the modern architecture used to build an application in the project, as seen in Figure 23. Essentially, it is a system that is broken down into smaller components, where each can act independently and has a specific role. Such an approach is modular, which means it can be developed, deployed, and scaled independently. This means we can change the service without changing the whole system.

Docker containerization packages all dependencies, the runtime environment, and the code for each microservice. It ensures that each environment consistently executes the same way anywhere, making deployment and maintenance easier.

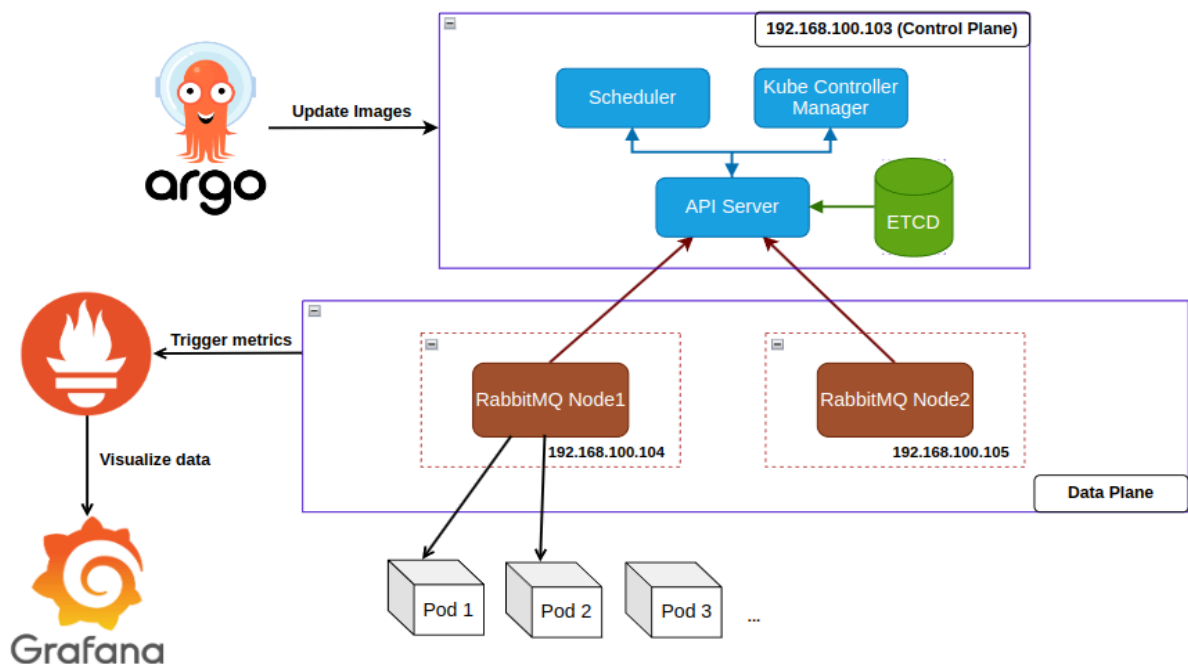


Figure 23. Kubernetes Cluster Architecture

The workflow is responsible for the building, testing, and packaging applications. On top of this, ArgoCD, the CD manager, takes the deployment process, where each new version of the Git repository is automatically updated and looks for content changes in the Git repository.

Kubernetes is a container orchestration platform that automates the application deployment, autoscaling, and administration of container applications. The project involves overseeing the distribution of microservices across several computers in a cluster by Kubernetes.

Ultimately, Prometheus serves as an application monitoring tool by collecting service metrics. The metrics generated by Grafana, a visualization tool, are then used in the dashboards to scrutinize the application's performance and health. This strategy aims to support the smooth operation of the microservices-based application from development to deployment and monitor users' activities.

3.2.5. Deployment and Orchestration

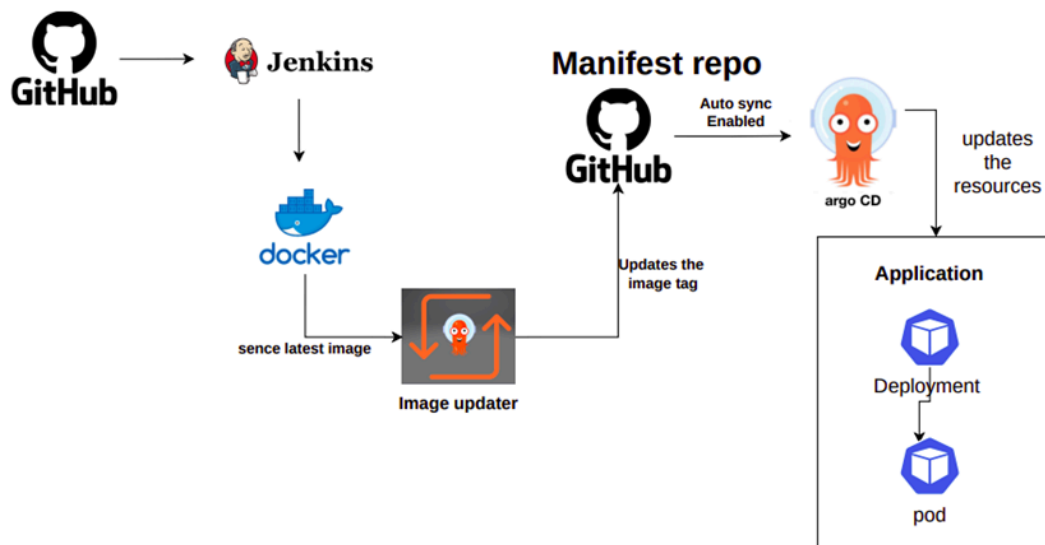


Figure 24. DevOps Integration of Kubernetes with Jenkins, Docker, and ArgoCD

We put the microservices, grammar detection and chapter summarization into their package, including everything Docker technology is required for.

However, we need a tool to manage these containers' placement, communication, and capacity issues. It is the place of Kubernetes where they come in. This powerful orchestration platform is a supersize for Docker containers' deployment, networking, and scaling across multiple servers, converting our containerized microservices into a well-coordinated system.

Here are the main components and their roles within this system in Figure 24:

- GitHub: Serves as the source code repository. Developers push their code changes here, which triggers the CI/CD pipeline.
- Jenkins: A continuous integration service that monitors the GitHub repository for changes. Once changes are detected, Jenkins initiates a build process that includes compiling the code and running tests.
- Docker: Jenkins uses Docker to containerize the application, ensuring that it can run uniformly across different environments. Docker packages the application with all its dependencies into a Docker image.
- Image Updater: A tool or script that detects when a new Docker image is available. It updates the image tag in the manifest repository on GitHub, ensuring that the deployment uses the latest image.
- Manifest Repo (GitHub): A separate repository on GitHub that contains the Kubernetes deployment manifests. This repository is set to auto-sync with Argo CD, which means any changes to the manifests are automatically detected and applied.
- Argo CD: A continuous delivery tool for Kubernetes that automatically synchronizes the specified state of applications deployed in Kubernetes to match the manifest files stored in the GitHub repository. It detects changes in the manifest repository and updates the Kubernetes resources accordingly.
- Application Deployment: The application is deployed as a set of pods within a Kubernetes cluster. The deployment is defined in the Kubernetes manifests, specifying how the application should be run, scaled, and managed.
- Feedback Loop: Any changes to the GitHub repositories (source code or manifests) are continuously integrated and delivered through this automated pipeline, allowing for rapid updates and iterative improvements to the application.

3.2.6. Communication and Data Flow

To function effectively, each element of the system can use the data and the feedback it delivers, making the system's process a feedback loop. This complex system is the key to this situation, with the help of the secure storage of the data and transferring it through the system.

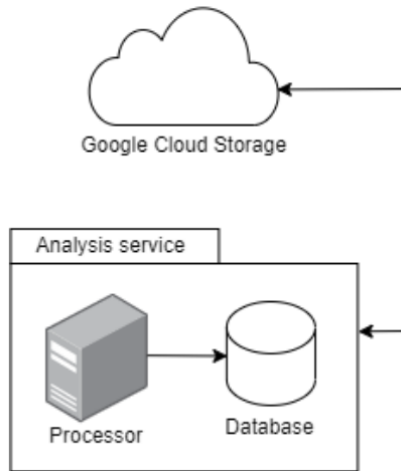


Figure 25. Google Cloud Storage Data Flow

The dissertations are stored in the Google Cloud Storage (GCS) as a scalable and secure vault. Each service event is directed to the specific storage of the particular thesis in GCS, making it convenient to access and analyze the work.

GCS enables the hassle-free communication of data and events. It serves as the main part of the system, thus ensuring fast and smooth operations. With the increase in the number of jobs, GCS can quickly add more distance without experiencing a shortage of available resources. Its distributed and asynchronous event stream is crucial to the real-time analysis of the situation and, thus, keeps the students and staff up to-date and involved in the thesis review procedure.

CHAPTER 4

IMPLEMENT AND RESULTS

This section is a walkthrough of various techniques, including initiating the servers and Kubernetes cluster environment by Jenkins and ArgoCD and treating the new NLP services inside the system. In the end, the objective is to establish a solid pipeline that can do all the thesis submission tasks. The pipeline's other strengths, such as scalability, high availability, and fault tolerance, are also commendable. The completion of the pipeline, which would be to conduct an actual lab assignment that shows the system's capabilities, is also placed in the last part of this chapter.

4.1. Implement

The system consists of four focal points: creating new services, implementing the Kubernetes cluster, visualizing and automating the Kubernetes cluster update on ArgoCD, and configuring Jenkins to create a pipeline for assessing thesis submissions.

4.1.1. Implementing Citation Service

The microservices architecture of this service is a state-of-the-art technology. Every individual service is a module bundled within a Docker container, and we can verify that using Figure 26. These rates are aware of each other and communicate smoothly through a message queue (RabbitMQ). The implementation method supported by DevOps is being followed to ensure its utility and robustness. The molar architecture makes the system very flexible in that it can handle new users and service accuracies at the same time.

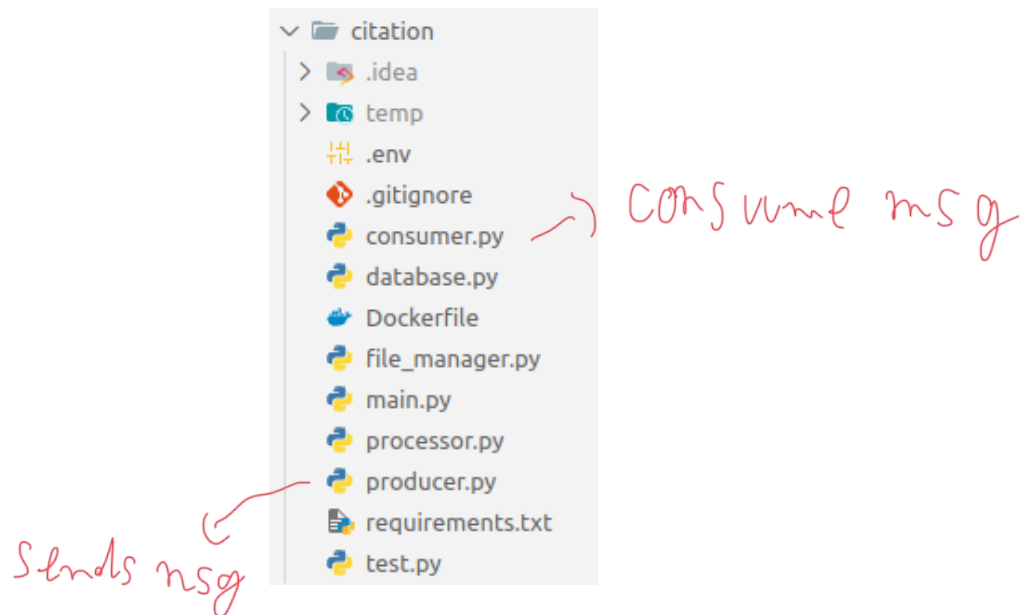


Figure 26. Citation Service

To be more specific, **consumer.py** is the file used to filter the incoming messages from other services and the message consumption from RabbitMQ queues. Consequently, **producer.py** sends the tasks and data to the designated RabbitMQ queues. Efficient data storage will be guaranteed by managing file uploads and downloads of Google Cloud Storage from **file_manager.py**. The duties of the service entail processing the tasks, if necessary, doing any of the math or data manipulations, and outputting the data to a file using the domain of **processor.py**. In the context of the microservices architecture, RabbitMQ itself also plays a significant role in effective coordination and communication among the different elements, thus improving the system's scalability and performance.

The **extract_citations** function helps format a file to include distinct citations and create a list of citations. Firstly, the *extract_references_from_file* function attempts to extract references from the presented document using the *refextract* library [14]. The *merge_citations* function controls the references found and is responsible for merging the appropriate citations.

```

FUNCTION extract_citations(uploaded_file_location)
    TRY
        citations = CALL extract_references_from_file(uploaded_file_location)

        citations = CALL merge_citations(citations)

        DECLARE unique_citations AS DICTIONARY

        FOR EACH citation IN citations
            raw_ref = citation['raw_ref'][0]
            IF raw_ref NOT IN unique_citations
                unique_citations[raw_ref] = citation
            ELSE
                CONTINUE

        unique_citations_list = CONVERT unique_citations.values() TO LIST

        RETURN unique_citations_list

    EXCEPT Exception AS e
        PRINT "An error occurred: " + e
        RETURN EMPTY LIST

```

Figure 27. Pseudocode of citation processing function

The method initializes an empty dictionary named *unique_citations* to guarantee the uniqueness of the citations. Then, it again goes through every citation in the combined citations list. It retrieves the raw reference string *raw_ref* for every citation and determines if it already exists as a key in the *unique_citations* dictionary. The citation is added to the dictionary using the raw reference as the key if the raw reference is not already there. The citation is omitted in cases when the raw reference is available previously.

After processing each citation, the function creates a list named *unique_citations_list* from the contents of the *unique_citations* dictionary. The function's output is returned as this list, which only includes unique citations. During this procedure, if an exception arises, it is caught, an error message is written, and an empty list is given back. This guarantees that the function can gracefully handle failures without resulting in a program crash.

Besides that, similar citations in a list may be combined using the **merge_citations** function to generate a consolidated list of citations, as shown in Figure 28. It first initializes an empty list named *merged_citations* to hold the merged citation data.

```

FUNCTION merge_citations(citations)

    INITIALIZE merged_citations AS EMPTY LIST

    FOR EACH citation IN citations
        raw_ref = citation['raw_ref'][0]

        ieee_pattern = REGEX PATTERN '^[\d+\\]'
        apa_pattern = REGEX PATTERN '.*\\(.*\\d{4}\\).*'

        is_new_ieee_citation = SEARCH ieee_pattern IN raw_ref
        is_new_apa_citation = SEARCH apa_pattern IN raw_ref

        IF merged_citations IS EMPTY OR is_new_ieee_citation OR
                                           is_new_apa_citation
            APPEND citation TO merged_citations
        ELSE
            APPEND raw_ref TO merged_citations[-1]['raw_ref'][0] WITH SPACE
            IF 'misc' IN citation
                IF 'misc' IN merged_citations[-1]
                    APPEND citation['misc'][0] TO merged_citations[-1]['misc'][0]
                                                                WITH SPACE
                ELSE
                    SET merged_citations[-1]['misc'] TO citation['misc']

    RETURN merged_citations

```

Figure 28. Pseudocode of citation merging function

Next, the function goes through each citation in the supplied list of citations one by one. It extracts the *raw_ref*, or the citation's raw reference string, for every citation. The function specifies two regular expression patterns: *apa_pattern* to identify citations in the APA style and *ieee_pattern* to detect citations in the IEEE style. The next step determines if the citation is a fresh IEEE or APA citation by comparing it to either of these patterns in the current *raw_ref*.

The function adds the current citation to the **merged_citations** list if it is not already there or if it is recognized as a new IEEE or APA citation. The function ensures that the references are concatenated with a space by appending the *raw_ref* of the current citation to the final item in **merged_citations** if the citation is not a new entry. Furthermore, in the event that a *misc* field is present in the current citation, the function verifies if the most recent item in **merged_citations** also has a *misc* field. If so, the current citation's *misc* field is added, separated by a space, to the previous entry's *misc* field. The *misc* field from the current citation is initialized if the previous entry does not have one.

The function returns the **merged_citations** list with the consolidated citation items after processing all of the citations.

```

FUNCTION check_citations(citations)

    INITIALIZE citation_styles AS EMPTY LIST

    FOR EACH citation IN citations
        IF is_website_apa_style(citation)
            APPEND ("APA Style (Website)", citation['raw_ref'][0]) TO citation_styles
        ELSE IF is_website_mla_style(citation)
            APPEND ("MLA Style (Website)", citation['raw_ref'][0]) TO citation_styles
        ELSE IF is_website_ieee_style(citation)
            APPEND ("IEEE Style (Website)", citation['raw_ref'][0]) TO citation_styles
        ELSE IF is_apa_style(citation)
            APPEND ("APA Style", citation['raw_ref'][0]) TO citation_styles
        ELSE IF is_mla_style(citation)
            APPEND ("MLA Style", citation['raw_ref'][0]) TO citation_styles
        ELSE IF is_ieee_style(citation)
            APPEND ("IEEE Style", citation['raw_ref'][0]) TO citation_styles
        ELSE
            APPEND ("Unknown Style", citation['raw_ref'][0]) TO citation_styles

    RETURN citation_styles

```

Figure 29. Pseudocode of citation checking function

The **check_citations** function (Figure 29) processes a list of citations and classifies each citation according to its style and structure. It initializes an empty list called *citation_styles* to hold tuples identifying the detected citation style and the matching raw reference text for each citation.

The function iterates over every citation in the *citations* list. It determines the categorization of each reference by comparing it to a number of style-checking functions, as shown in Figure 30:

- **is_website_apa_style**: Checks if the citation matches APA style with a website link.
- **is_website_mla_style**: Checks if the citation matches MLA style with a website link.
- **is_website_ieee_style**: Checks if the citation matches IEEE style with a website link.
- **is_apa_style**: Checks if the citation matches APA style without a website link.
- **is_mla_style**: Checks if the citation matches MLA style without a website link.
- **is_ieee_style**: Checks if the citation matches IEEE style without a website link.
- If none of these patterns match, the citation is categorized as "Unknown Style".

```

def is_apa_style(citation):
    return re.match(r".*\d{4}\.", citation['raw_ref'][0])

def is_website_apa_style(citation):
    return re.match(r".*\d{4}\.(?:http|https)://.*", citation['raw_ref'][0])

def is_mla_style(citation):
    return re.match(r".*\d{4}\.", citation['raw_ref'][0])

def is_website_mla_style(citation):
    return re.match(r".*\d{4}\.(?:http|https)://.*", citation['raw_ref'][0])

def is_ieee_style(citation):
    return re.match(r".*[\d+\.]*(?:\s+[Online]\s+Available:\s+S+)?", citation['raw_ref'][0])

def is_website_ieee_style(citation):
    return re.match(r".*[\d+\.]*(?:http|https)://.*", citation['raw_ref'][0])

```

Figure 30. Style-checking functions

Each style-checking function uses regular expressions to match against the *raw_ref* field of the citation, which is assumed to be a string containing the reference information.

After evaluating the citation against all style-checking functions, the function appends a tuple

[style_category, raw_reference]

to the *citation_styles* list, where *style_category* represents the detected citation style (e.g., "APA Style," "MLA Style," "IEEE Style," "Unknown Style") and *raw_reference* is the original reference string from the citation.

Finally, the function returns the *citation_styles* list, which contains tuples representing the detected citation styles and their respective raw reference strings. This allows for further analysis or processing based on the categorized citation styles.

4.1.2. Implementing Table and Figure Detection Service

Like the Citation service, the Table and Figure Detection service adheres to a modular and directory structure of Event-driven architecture, as shown in Figure 31. RabbitMQ is also a central communication hub, facilitating seamless interaction between individual services encapsulated within Docker containers.

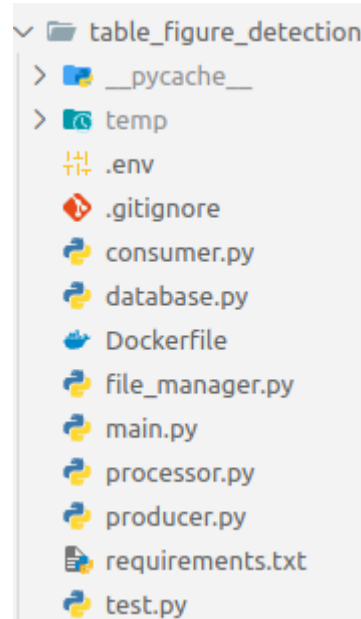


Figure 31. Table/Figure Detection Service

Armed with the precision of regular expressions and the elegance of text analysis algorithms, the service meticulously scours uploaded PDF reports to pinpoint table and figure titles, regardless of location. No image escapes its detection, be it nestled within the main text, adorning table headings, or even tucked away in figure captions.

```

FUNCTION extract_figure_titles(uploaded_file_location,
                               ignore_pages_with=['contents', 'figures'], exclude_phrases=[])
  OPEN pdfplumber document from uploaded_file_location AS pdf
  INITIALIZE figure_titles AS empty list
  INITIALIZE figure_title_pattern AS regex pattern 'Figure \d+'

  FOR EACH page in pdf.pages WITH INDEX i
    EXTRACT text from page AND convert to lowercase

    IF any keyword in ignore_pages_with IS IN text
      CONTINUE to next iteration # Skip pages like table of contents,
                                list of figures, etc.

    EXTRACT figures from page

    FOR EACH figure in figures
      DEFINE title_region as region around the bottom of the figure
      EXTRACT title_crop from page within title_region
      EXTRACT title_text from title_crop

      IF title_text EXISTS
        CLEAN title_text by splitting at '\n' AND stripping
                                                whitespace

        IF clean_title MATCHES figure_title_pattern
          IF none of the phrases in exclude_phrases IS IN
                                                    clean_title
            APPEND clean_title TO figure_titles

  RETURN figure_titles

```

Figure 32. Pseudocode of extract_figure_titles function

The **extract_figure_titles** function aims to extract titles of figures from a PDF document located at *uploaded_file_location*, as shown in Figure 32. It skips pages that likely contain table of contents or list of figures based on provided keywords (*ignore_pages_with*). It also excludes titles containing specific phrases (*exclude_phrases*). The function uses the *pdfplumber* library to open the PDF and iterate through each page. For each page, it looks for images (assumed to be figures) and extracts text near the bottom of each image to capture potential titles. Titles matching the pattern "Figure \d+" are collected into the *figure_titles* list, which is returned as the output.

```

FUNCTION extract_table_titles(uploaded_file_location,
                             ignore_pages_with=['contents', 'tables'], exclude_phrases=[])

  OPEN PDF file at uploaded_file_location WITH pdfplumber
  INITIALIZE table_titles AS EMPTY LIST

  DEFINE table_title_pattern AS REGEX PATTERN '^Table \d+'

  FOR EACH page IN pdf.pages
    text = EXTRACT text FROM page
    IF text IS NOT PRESENT
      CONTINUE

    IF ANY keyword IN ignore_pages_with IS IN text (CASE-INSENSITIVE)
      CONTINUE # Skip pages likely to be table of contents, list of
                                                         tables, etc.

    FOR EACH line IN SPLIT text BY '\n'
      IF table_title_pattern.search(line.strip())
        title = line.strip()

        IF NOT ANY phrase IN exclude_phrases IS IN title
          (CASE-INSENSITIVE)
          APPEND title TO table_titles

    tables = EXTRACT tables FROM page

    FOR EACH table IN tables
      IF table IS NOT EMPTY
        title_row = table[0]
        CONVERT title_row ITEMS TO STRINGS AND REPLACE NONE WITH
                                                         EMPTY STRING
        title_text = JOIN title_row ITEMS BY SPACE AND STRIP

        IF title_text IS PRESENT
          IF table_title_pattern.search(title_text)
            IF NOT ANY phrase IN exclude_phrases IS IN title_text
              (CASE-INSENSITIVE)
              APPEND title_text TO table_titles

  RETURN table_titles

```

Figure 33. Pseudocode of *extract_table_titles* function

Similarly, in the Figure 33, the **extract_table_titles** function operates similarly to **extract_figure_titles**, but focuses on extracting titles of tables from the PDF document. It skips pages with contents or tables based on provided keywords (*ignore_pages_with*) and excludes titles containing specific phrases (*exclude_phrases*). The function iterates through

each page and searches for lines matching the pattern "Table \d+" to identify table titles. Additionally, it extracts tables using *pdfplumber*'s *extract_tables()* method and processes the first row of each table to extract potential table titles. Titles meeting the criteria are added to the *table_titles* list, which is returned as the output.

Furthermore, every discovered title and reference is meticulously tagged with the corresponding page number, transforming the report into a hyperlinked landscape (Figure 34). Researchers can effortlessly access the source of any visual insight with a single click, eliminating the need for tedious scrolling and manual page searching.

```

FUNCTION find_figure_mentions(uploaded_file_location, figure_titles)

    OPEN PDF file at uploaded_file_location WITH pdfplumber
    INITIALIZE figure_mentions AS DICTIONARY WITH KEYS FROM figure_titles AND
                                                EMPTY LISTS AS VALUES

    FOR EACH page IN pdf.pages WITH INDEX STARTING FROM 1
        text = EXTRACT text FROM page
        IF text IS NOT PRESENT
            CONTINUE

        FOR EACH title IN figure_titles
            pattern = CREATE CASE-INSENSITIVE REGEX PATTERN FROM title
            matches = FIND ALL INSTANCES OF pattern IN text
            IF matches IS NOT EMPTY
                APPEND (NUMBER OF matches, page INDEX) TO
                    figure_mentions[title]

    RETURN figure_mentions

FUNCTION find_table_mentions(uploaded_file_location, table_titles)

    OPEN PDF file at uploaded_file_location WITH pdfplumber
    INITIALIZE table_mentions AS DICTIONARY WITH KEYS FROM table_titles AND
                                                EMPTY LISTS AS VALUES

    FOR EACH page IN pdf.pages WITH INDEX STARTING FROM 1
        text = EXTRACT text FROM page
        IF text IS NOT PRESENT
            CONTINUE

        FOR EACH title IN table_titles
            pattern = CREATE CASE-INSENSITIVE REGEX PATTERN FROM title
            matches = FIND ALL INSTANCES OF pattern IN text
            IF matches IS NOT EMPTY
                APPEND (NUMBER OF matches, page INDEX) TO
                    table_mentions[title]

    RETURN table_mentions

```

Figure 34. Pseudocode of *find_figure_mentions* and *find_table_mentions* functions

The ***find_figure_mentions*** tool looks for references to figures, formerly extracted titles, all over the PDF page. It maps every figure title to an empty list, starting with a dictionary figure mentioned. Searching for instances of each figure title using case-insensitive matching, the function runs throughout every page of the PDF. Along with the page number where they happen, matches are noted. The function generates figure references, a dictionary

in which every key is a figure title, and the value is a set of tuples showing the number of mentions and the matching page counts.

Similarly, the **find_table_mentions** tool looks for references to previously obtained titles, that is, tables, all over the PDF file. It maps every table title to an empty list, starting a dictionary table mentioning system. Searching for instances of each table title using case-insensitive matching, the function runs over every page of the PDF. Along with the page number where they happen, matches are noted. The function generates table references, a dictionary in which every key is a table title, and the value is a set of tuples showing the number of mentions and related page numbers.

4.1.3. Implementing Grammar Service

The Grammar service also follows an Event-driven modular and directory-based design, much like the Citation and Table/Figure Detection services.

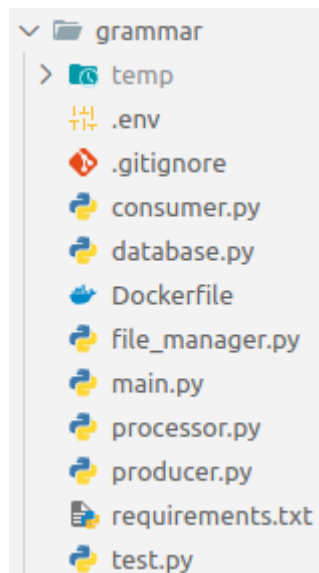


Figure 35. Grammar Service

However, the service's true power lies in Natural Language Processing (NLP). Utilizing the advanced algorithms of the `language_tool_python` package, the service goes beyond basic spell checks (Figure 36). It meticulously examines uploaded reports, analyzing grammar, style, and clarity in intricate detail. This empowers the service to identify sentence structure issues and stylistic errors and suggests intelligent improvements for enhanced readability.

```

FUNCTION check_grammar(uploaded_file_location)
    TRY
        INITIALIZE tool WITH language_tool_python FOR 'en-US'

        OPEN PDF file at uploaded_file_location WITH pdfplumber
        INITIALIZE full_text AS EMPTY STRING

        FOR EACH page IN pdf.pages
            EXTRACT text FROM page
            APPEND extracted text AND NEW LINE TO full_text

        matches = CALL tool.check(full_text)

        RETURN matches, full_text

    EXCEPT Exception AS e
        PRINT "An error occurred: " + e
        RETURN EMPTY LIST

```

Figure 36. Pseudocode of `check_grammar` functions

Designed to use the *language_tool_python* package, the **check_grammar** function is meant to grammar check the text taken from a PDF file. The function starts with setting an English (US) grammar checking language tool. It then opens the PDF file indicated by the *uploaded_file_location* using *pdfplumber*.

Originally empty, *full_text* is set to include the concatenated text from every PDF page. The function gathers the text content and appends a newline character to *full_text* for every page in the PDF. This mechanism guarantees that one single string gathers the text of the whole page.

The function checks the grammar of the gathered text using the language tool after text extraction from all pages. containing *full_text*, the *tool.check* method is invoked and the output, *matches*, containing grammatical check results, is kept.

At last, the function produces a tuple including the *full_text* and the *matches*. Should any problem arise during this procedure, an error message prints and the method generates an empty list.

4.1.4. Implementing the Kubernetes Cluster

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: word-frequency-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: word-frequency
  template:
    metadata:
      labels:
        app: word-frequency
    spec:
      volumes:
        - name: google-credentials
          secret:
            secretName: google-credentials
      containers:
        - name: word-frequency
          image: daniel135dang/word_frequency:1.0.0-16
          ports:
            - containerPort: 6010
          volumeMounts:
            - name: google-credentials
              mountPath: /app/google_credentials.json
              subPath: google_credentials.json
              readOnly: true
          env:
            - name: APP_NAME
              value: "word_frequency"
            - name: POSTGRES_NAME
              value: "word_frequency"
          envFrom:
            - configMapRef:
                name: services-config
```

Figure 37. A sample Deployment of a service

In the provided Kubernetes deployment manifest (Figure 37), we outline the configuration for deploying a containerized application on a Kubernetes cluster. This configuration is generic and can be adapted for deploying various services by modifying certain parameters, such as the application name and image. Below, we provide a detailed breakdown of each section of the deployment manifest:

- The *apiVersion* specifies the version of the Kubernetes API that is being used, in this case, *apps/v1*, which is standard for defining deployments. The *kind: Deployment*

indicates that this manifest is for deploying a set of pods managed by a deployment controller.

- The *metadata* section includes the *name* field, which specifies the name of the deployment. This name is used to identify the deployment within the Kubernetes cluster.
- The *spec* section outlines the desired state for the deployment. This includes the number of replicas. If it is set to 1, one instance of the application will be running.

The *selector* field specifies how to identify the pods managed by this deployment. It uses `matchLabels` to match pods with the label *app*: `<service-name>`.

- The *template* section describes the pods that will be created by the deployment.
 - The *metadata* subsection within the template includes labels that are applied to the pods, ensuring they match the selector defined earlier.
 - The *spec* subsection within the template defines the configuration for the containers within the pods.
- The *volumes* field defines a volume named `google-credentials`. This volume sources its data from a Kubernetes secret named `google-credentials`.
- The *containers* field defines the container specifications.
 - The *name* field specifies the name of the container.
 - The *image* field specifies the container image to be used.
 - The *ports* field defines which port the container exposes.
- The *volumeMounts* field within the container definition specifies that the *google-credentials* volume should be mounted at `/app/google_credentials.json` within the container. The *subPath* field ensures that only the *google_credentials.json* file from the secret is mounted.
- The *env* field defines environment variables that are passed into the container.

The *envFrom* field allows environment variables to be sourced from a `ConfigMap`, specified by the *configMapRef* field. This `ConfigMap` contains additional configuration data that the application may require.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: services-config
data:
  RABBITMQ_HOST: "rabbitmq"
  RABBITMQ_PORT: "5672"
  RABBITMQ_USER: "guest"
  RABBITMQ_PASSWORD: "guest"
  POSTGRES_HOST: "postgresql"
  POSTGRES_PORT: "5432"
  POSTGRES_USER: "postgres"
  POSTGRES_PASSWORD: "123456"
  POSTGRES_DB: "thesis_upload"
  GOOGLE_CLOUD_STORAGE_BUCKET: "thesis_file_bucket"
  ROOT_DIR: "/app"
```

Figure 38. ConfigMap to store non-confidential data

In addition to the deployment manifest, we have a Kubernetes ConfigMap manifest that defines configuration data for the application, as shown in Figure 38. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable:

- The *apiVersion* is set to *v1*, indicating the version of the Kubernetes API being used. The *kind* is *ConfigMap*, signifying that this manifest defines a ConfigMap resource.
- The *metadata* section includes a *name* field which specifies the name of the ConfigMap. This name is used to reference the ConfigMap within other Kubernetes resources, such as deployments or pods.
- The *data* section contains key-value pairs representing configuration settings for the application. Each key corresponds to a configuration parameter that the application uses.
 - RABBITMQ_HOST specifies the hostname of the RabbitMQ service.
 - RABBITMQ_PORT specifies the port number on which RabbitMQ is listening.
 - RABBITMQ_USER and RABBITMQ_PASSWORD provide the credentials for connecting to RabbitMQ.
 - POSTGRES_HOST specifies the hostname of the PostgreSQL database service.
 - POSTGRES_PORT specifies the port number on which PostgreSQL is listening.

- `POSTGRES_USER` and `POSTGRES_PASSWORD` provide the credentials for connecting to PostgreSQL.
- `POSTGRES_DB` specifies the name of the PostgreSQL database to be used.
- `GOOGLE_CLOUD_STORAGE_BUCKET` specifies the name of the Google Cloud Storage bucket.
- `ROOT_DIR` specifies the root directory path for the application within the container.

Using a *ConfigMap*, we can keep the application coding apart from the container images, making configuration management more flexible and dynamic.

The *envFrom* field in the container specification connects to the *services-config* field shown in the deployment manifest. This lets the container access the environment variables specified in the ConfigMap during execution.

With this mechanism in place, the application is ensured to be given the needed configuration information, such as the database passwords and the service endpoints, which can be accessed without the need to hardcode these values into the container image. Besides such quality, this either produces better security procedures or a more simplified configuration that goes through different settings.


```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgresql
  labels:
    app: postgresql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgresql
  serviceName: postgresql
  template:
    metadata:
      labels:
        app: postgresql
    spec:
      tolerations:
        - key: "node-role.kubernetes.io/control-plane"
          operator: "Exists"
          effect: "NoSchedule"
      containers:
        - name: postgresql
          image: daniel1135dang/postgresql:1.0.0-6
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/postgres/data
          env:
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: services-config
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                configMapKeyRef:
                  name: services-config
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                configMapKeyRef:
                  name: services-config
                  key: POSTGRES_PASSWORD
      volumeClaimTemplates:
        - metadata:
            name: postgres-data
          spec:
            accessModes:
              - ReadWriteOnce
            storageClassName: standard
            resources:
              requests:
                storage: 10Gi

```

Figure 39. PostgreSQL StatefulSet

More than that, stateful sets are tailored to bring pod sets up and down and simultaneously ensure the identity of each pod will be sticky, as illustrated in Figure 39. This is particularly the case in the case of applications like databases that require persistent storage and stable network identities:

- The *apiVersion* is set to *apps/v1*, indicating the version of the Kubernetes API being used. The kind is *StatefulSet*, signifying that this manifest defines a StatefulSet resource.
- The *metadata* section includes a *name* field that specifies the name of the StatefulSet and *labels* that help identify and manage the StatefulSet within the Kubernetes cluster.
- The *spec* section outlines the desired state for the StatefulSet. This includes:
 - *replicas*: Specifies the number of pod replicas to be maintained.
 - *selector*: Specifies how to identify the pods managed by this StatefulSet using *matchLabels*.
 - *serviceName*: Specifies the name of the headless service that governs the StatefulSet.
- The *template* section describes the pods that will be created by the StatefulSet.
 - The *metadata* subsection within the template includes labels that are applied to the pods, ensuring they match the selector defined earlier.
 - The *spec* subsection within the template defines the configuration for the containers within the pods.
- The *tolerations* field specifies conditions under which the pods can be scheduled onto nodes. In this case, it allows the pods to be scheduled on control-plane nodes by tolerating the *NoSchedule* taint.
- The *containers* field defines the container specifications.
 - *name*: Specifies the name of the container.
 - *image*: Specifies the container image to be used.
 - *ports*: Defines the port that the container exposes.
 - *volumeMounts*: Specifies the volume to be mounted inside the container.
- The *env* field defines environment variables that are passed into the container. These variables are sourced from the *services-config* ConfigMap:
 - *POSTGRES_DB*: Specifies the name of the PostgreSQL database.
 - *POSTGRES_USER*: Specifies the username for PostgreSQL.
 - *POSTGRES_PASSWORD*: Specifies the password for PostgreSQL.
- The *volumeClaimTemplates* section defines a PersistentVolumeClaim (PVC) template used by the StatefulSet.
 - *metadata*: Includes the name of the PVC.

- *spec*: Specifies the details of the storage request.
 - *accessModes*: Indicates the access mode for the volume.
 - *storageClassName*: Specifies the storage class to be used.
 - *resources*: Defines the storage resources required.

The StatefulSet uses ConfigMap's *services-config* feature to get Postgres configuration data. This guarantees that the database setup is handled outside from the container image, therefore encouraging improved security standards and simpler configuration management throughout many environments.

4.1.5. Implementing Kubernetes Cluster on ArgoCD



Figure 40. Kubernetes Cluster visualized on ArgoCD

Implementing ArgoCD on-premise solution necessitates meticulous planning. The first step involves setting up a suitable VM server with ample processing power, secure network configurations, and dedicated storage to house the TRMS. This ensures a reliable and safe environment for the system to operate.

After the initial configuration, the Kubernetes cluster takes center stage. Every essential part of the cluster is precisely adjusted and fine-tuned to meet the specific requirements of the TRMS. Kubernetes orchestrates the deployment and management of all TRMS services, facilitating seamless and efficient operation.

After finishing the deployment of Kubernetes, ArgoCD implements automated deployment routines, streamlining the process of updating and deploying new versions of the TRMS (Figure 40). Additionally, ArgoCD establishes a connection to the TRMS's Git repository, allowing for continuous monitoring and updates. This ensures the TRMS remains current and functions at its best.

4.1.6. Configure Jenkins

After the containers hold functions such as figure identification, grammar checker, and citation manager are carefully constructed, Jenkins releases them into the Docker Hub stage so that they may be quickly deployed to the Kubernetes infrastructure, as shown in Figure 41.

```
stage('Build & Push Docker Images') {
    steps {
        script {
            // Get a list of all directories in the current workspace
            def output = sh(script: "find . -maxdepth 1 -type d", returnStdout: true).trim()

            // Split the output into a list of directories
            def services = output.split("\n").collect { it.replace("./", "") }

            // List of directories to exclude
            def excludeServices = ['rabbitmq', 'readme_images', 'requirements', '.git', '.', '.idea', '.scannerwork']

            for (service in services) {
                // Skip the current iteration if the service is in the exclude list
                if (excludeServices.contains(service)) {
                    continue
                }

                echo "Processing service: ${service}"

                def imageName = "daniel135dang/${service}"

                def builtImage = docker.build(imageName, "./${service}")

                withDockerRegistry(credentialsId: 'dockerhub', url: 'https://index.docker.io/v1/') {
                    builtImage.tag("${IMAGE_TAG}")
                    docker.image("${imageName}:${IMAGE_TAG}").push()
                }
            }
        }
    }
}
```

Figure 41. A stage of the Jenkins pipeline

Jenkins ensures there are no security issues and maintains an orderly atmosphere for the production's seamless flow by tidying up temporary artifacts when the stage has been cleared and the instruments adjusted.

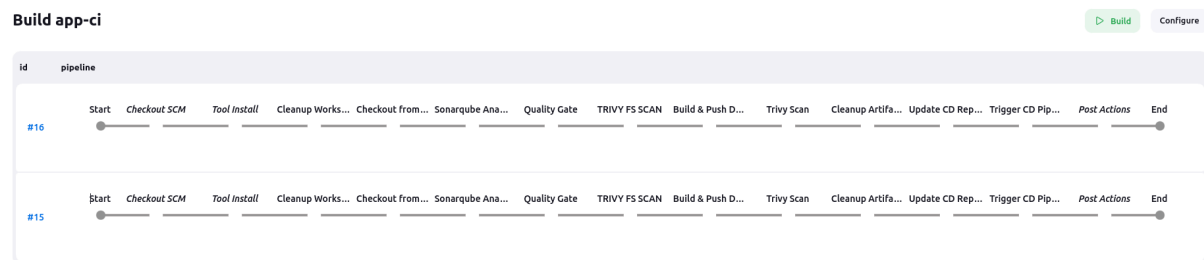


Figure 42. Comprehensive Jenkins pipeline

Jenkins finally starts the continuous delivery pipeline, the next act (Figure 42). Based on that, the TRMS guarantees a smooth transfer from development to production. Jenkins, the ever-dedicated director, muses and makes observations following the show. This creates an extensive report sent via email, akin to a post-show debrief, praising a job well done or outlining development opportunities.

4.2. Results

Figure 43. Login Page

When accessing the website, the first page displayed is the authentication page, which asks for the user's credentials, as shown in Figure 43. The required information includes entering the user's ID code for authentication and authorization. The default password will be the user's date of birth in the form "DDMMYYYYY." Then, the corresponding page appears.

Thesis Submission

Deadline: 12:00AM 01/12/2024

Submit your thesis document before the deadline. Receive reports on various aspects and improve your work.

Thesis title

Advisor

File

 No file chosen

Figure 44. Submission Page

The homepage in Figure 44 will be shown once the user logs in as a student. A thesis submission form will appear if they have never submitted their paper. The form asks for the thesis title, has a drop-down menu with a selection of advisers, and only accepts PDF files for file submission. Additionally, extra information, such as the deadline, is added to remind students.

After completing the feedback generation process, services will no longer display a spinner animation and can be chosen. Their findings are shown as an exclamation point if there was a processing fault, a cross if the submission did not match the guidelines, and a tick if it did. Because analytical services are not created according to criteria, they do not yield outcomes.

test

Dang Nhat Huy - ITITI20043

Submitted at: 8:49PM 18/06/2024

Download file

Download report

Guidelines

Resubmit

Template-based		Chapter titles according to template:
Chapter title	✓	ABSTRACT
Format check	✓	CHAPTER 1 INTRODUCTION
Page count	✓	CHAPTER 2 LITERATURE REVIEW/RELATED WORK
Table of content	✓	CHAPTER 3 IMPLEMENTATIONS
Citation	✓	CHAPTER 4 IMPLEMENT AND RESULTS
Table figure detection	✓	CHAPTER 5 DISCUSSION AND EVALUATION
Grammar	✓	CHAPTER 6 CONCLUSION AND FUTURE WORK
Analytical		REFERENCES
Word frequency		APPENDIX
Chapter summarization		Chapter titles detected in document:
		ABSTRACT
		CHAPTER 1 INTRODUCTION
		CHAPTER 2 BACKGROUND
		CHAPTER 3 METHODOLOGY
		CHAPTER 4 IMPLEMENT AND RESULTS
		CHAPTER 5 EVALUATION AND DISCUSSION
		CHAPTER 6 CONCLUSION AND FUTURE WORK
		REFERENCES



Figure 45. Thesis Report Management's dashboard

Figure 45 shows the analysis page after submitting it. Students can verify their completion status by selecting an analysis option from the left. The two types of services are analytical and template-based. As the name implies, template-based services provide feedback based on the University's established template. Analytical services offer perspectives and further input about the proposal.

The citation service will automatically extract the citation in the report and determine what type it belongs to, as shown in Figure 46.

Template-based	Chapter title ✓ Format check ✓ Page count ✓ Table of content ✓ Citation ✓ Table figure detection ✓ Grammar ✓	Analytical	Word frequency Chapter summarization
-----------------------	---	-------------------	---

List of Citations:

Correct Citations:

[1] T. STAUBITZ, H. KLEMENT, J. RENZ, R. TEUSNER, AND C. MEINEL, "Towards practical programming exercises and automated assessment in Massive Open Online Courses," in 2015 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE), Zhuhai, China: IEEE, Dec. 2015, pp. 23–30. doi: 10.1109/TALE.2015.7386010. (Type: IEEE Style)

[2] S. GROVER, P. FRANZ, E. SCHNEIDER, AND R. PEA, "The MOOC as distributed intelligence: Dimensions of a framework & evaluation of MOOCs," 2013. (Type: IEEE Style)

[3] C. BRESNAHAN AND R. BLUM, Linux essentials, Second edition. Indianapolis, Ind: Sybex, a Wiley brand, 2015. (Type: IEEE Style)

[4] K. MATTHIAS AND S. P. KANE, Docker: up and running, First edition. Sebastopol, CA: O'Reilly Media, Inc, 2015. (Type: IEEE Style)

[5] "Docker overview," Docker Documentation, 42:25 + +0200 200AD. <https://docs.docker.com/get-started/overview/> (accessed Aug. 29, 2023). (Type: IEEE Style (Website))

[6] B. BURNS, J. BEDA, AND K. HIGHTOWER, Kubernetes up & running: dive into the future of infrastructure, Second edition. Beijing [China] ; Boston [MA]: O'Reilly Media, 2019. (Type: IEEE Style)

[7] "Communication between Nodes and the Control Plane," Kubernetes. <https://kubernetes.io/docs/concepts/architecture/control-plane-node-communication/> (accessed Aug. 29, 2023). (Type: IEEE Style (Website))

[8] B. LASTER, JENKINS 2: up and running: evolve your deployment pipeline for nextgeneration automation, First edition. Sebastopol, CA: O'Reilly Media, 2018. (Type: IEEE Style)

[9] "Pipeline," Pipeline. <https://www.jenkins.io/doc/book/pipeline/> (accessed Aug. 29, 2023). (Type: IEEE Style (Website))

[10] "Build software better, together," GitHub. <https://github.com> (accessed Aug. 29, 2023). (Type: IEEE Style (Website))

[11] "Gradle Build Tool." <https://gradle.org/> (accessed Aug. 29, 2023). (Type: IEEE Style (Website))

[12] "How To Setup Jenkins On Kubernetes Cluster: Easy Guide," Jan. 12, 2021. <https://devopscube.com/setup-jenkins-on-kubernetes-cluster/> (accessed Jul. 17, 2023). (Type: IEEE Style (Website))

[13] C. Rodriguez-Bustos and J. Aponte, "How Distributed Version Control Systems impact open source software projects," in 2012 9th IEEE Working Conference on Mining Software Repositories (MSR), Zurich: IEEE, Jun. 2012, pp. 36–39. doi: 10.1109/MSR.2012.6224297. 59 (Type: IEEE Style)

No Issues found in citations.

Grade: 100%

Service Result: Pass

Figure 46. Citation service output

The table figure detection service will extract the figures and tables listed in the references section and then look up their location to determine whether the student has listed the correct page number of the figure/table as shown in the table of contents.

test

Dang Nhat Huy - ITITI20043

Submitted at: 9:16PM 18/06/2024

Download file

Download report

Guidelines

Resubmit

Template-based

Chapter title

Format check

Page count

Table of content

Citation

Table figure detection

Grammar

Analytical

Word frequency

Chapter summarization

Tables mentioned:

Figure 2.1: Linux mascot

Mentioned on page 13

Figure 2.2: Linux distros

Mentioned on page 13

Figure 2.3: Linux file system

Mentioned on page 14

Figure 2.4: Linux command-line interface

Mentioned on page 14

Figure 2.5: Docker icon

Mentioned on page 15

Figure 2.7: Some Docker components

Mentioned on page 17

Figure 2.8: Docker Anthology

Mentioned on page 17

Figure 2.9: Kubernetes icon

Mentioned on page 18

Figure 2.11: Pod in Kubernetes

Mentioned on page 21

Figure 2.12: Service in Kubernetes

Mentioned on page 21

Figure 47. Figure/Table detection service output

77

The grammar checking service will automatically check for errors related to grammar and typos in the thesis and then give reasons. Finally, suggest what students can do to correct the sentence.

Template-based	
Chapter title	✓
Format check	✓
Page count	✓
Table of content	✓
Citation	✓
Table figure detection	✓
Grammar	✓
Analytical	
Word frequency	
Chapter summarization	

Context: VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY THE INTERNATIONAL UNIVERSITY SCHO...
 Category: TYPOS
 Suggested correction: ['Ho Chi Minh', 'Hồ Chí Minh']

Context: ... SYSTEM FOR ASSIGNMENTS EVALUATION WITH DEVOPS A thesis submitted to the School of Com...
 Category: TYPOS
 Suggested correction: ['DevOps', 'DeVos', 'DEMONS', 'DEVELOPS', 'DROPS', 'DEVON', 'DEMOS', 'DEVOTE', 'DEVOUT', 'DEPOTS', 'DEVILS', 'DEVOID', 'DEVIOUS', 'DEVOTES', 'DEVOUR', 'DECOYS', 'DEVOURS', 'DECORS', 'DEEPS', 'DROOPS', 'DIVOTS', 'DEJEPS', 'DAVOS', 'EOPS', 'BEBOPS', 'DEFOGS', 'DEMOBS', 'DEPS', 'DEVORÉ', 'DEVORÉS', 'DEVs']

Context: ... Engineering Supervisor: Dr. Tran Thanh Tung Student name: Bui Minh Quang Student ID...
 Category: TYPOS
 Suggested correction: ['Sung', 'Tune', 'Hung', 'Lung', 'Jung', 'Tug', 'Tuna', 'Rung', 'Tang', 'TNG', 'Ting', 'Stung', 'Mung', 'Tong', 'Bung', 'Tuns', 'TUG', 'TUN', 'Dung', 'Tun']

Context: ...isor: Dr. Tran Thanh Tung Student name: Bui Minh Quang Student ID: ITITI19044 Ho C...
 Category: TYPOS
 Suggested correction: ['Boo', 'BUI', 'But', 'Book', 'Bus', 'Buy', 'Boom', 'Boot', 'Bug', 'Bud', 'GUI', 'DUI', 'Hui', 'Boon', 'Bun', 'Boos', 'Bum', 'BRI', 'BWI', 'Bur', 'Boor', 'Bub', 'AUI', 'BBI', 'BCI', 'BDI', 'BEI', 'BFI', 'BGI', 'BHI', 'BII', 'BJI', 'BKI', 'BLI', 'BMI', 'BNI', 'BOI', 'BPI', 'BSI', 'BTI', 'BU', 'BUA', 'BUB', 'BUC', 'BUD', 'BUE', 'BUF', 'BUH', 'BUJ', 'BUK', 'BUL', 'BUM', 'BUO', 'BUP', 'BUQ', 'BUR', 'BUT', 'BUU', 'BUV', 'BUW', 'BUX', 'BUY', 'BVI', 'BYI', 'Bi', 'NUI', 'OUI', 'Sui', 'TUI', 'UI', 'Boob', 'Bu']

Context: ... Tran Thanh Tung Student name: Bui Minh Quang Student ID: ITITI19044 Ho Chi Minh Cit...
 Category: TYPOS
 Suggested correction: ['Huang', 'Quant', 'Quango']

Context: ... SYSTEM FOR ASSIGNMENTS EVALUATION WITH DEVOPS APPROVED BY: _____
 Category: TYPOS
 Suggested correction: ['DevOps', 'DeVos', 'DEMONS', 'DEVELOPS', 'DROPS', 'DEVON', 'DEMOS',

Figure 48. Grammar checking service output

To facilitate the straightforward removal of services, the CI process will navigate through a collection of directories that house services within the repository. It will then compare with prior version controls to identify any modifications.

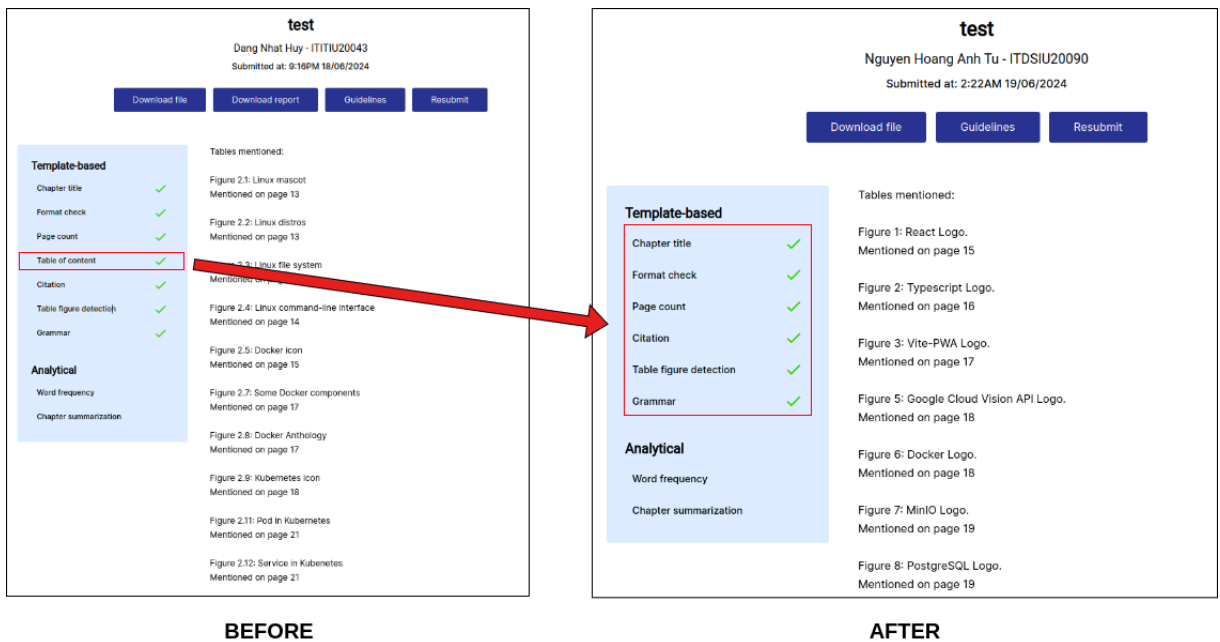


Figure 49. Example of removing Table of content service

For instance, if the `table_of_contents` has been deleted, the CI will automatically detect this change and initiate a process to update the manifest deployment file, thereby omitting the deleted service during deployment, as shown in Figure 49.

Additionally, for efficient tracking of removed services, we will catalog the names of these services in a file named `service_removed.txt`.

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Remove Services)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] readFile
[Pipeline] writeFile
[Pipeline] echo
Removed service: table-of-content from deployment.yaml
[Pipeline] readFile
[Pipeline] writeFile
[Pipeline] echo
Removed service: table-of-content from service.yaml
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Add New Services)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] readFile
[Pipeline] readFile
[Pipeline] echo
```

Figure 50. Removing service in CD pipeline

Similarly, the CI procedure scans through designated directories containing service configurations within the repository. It checks against previous version controls to identify additions.

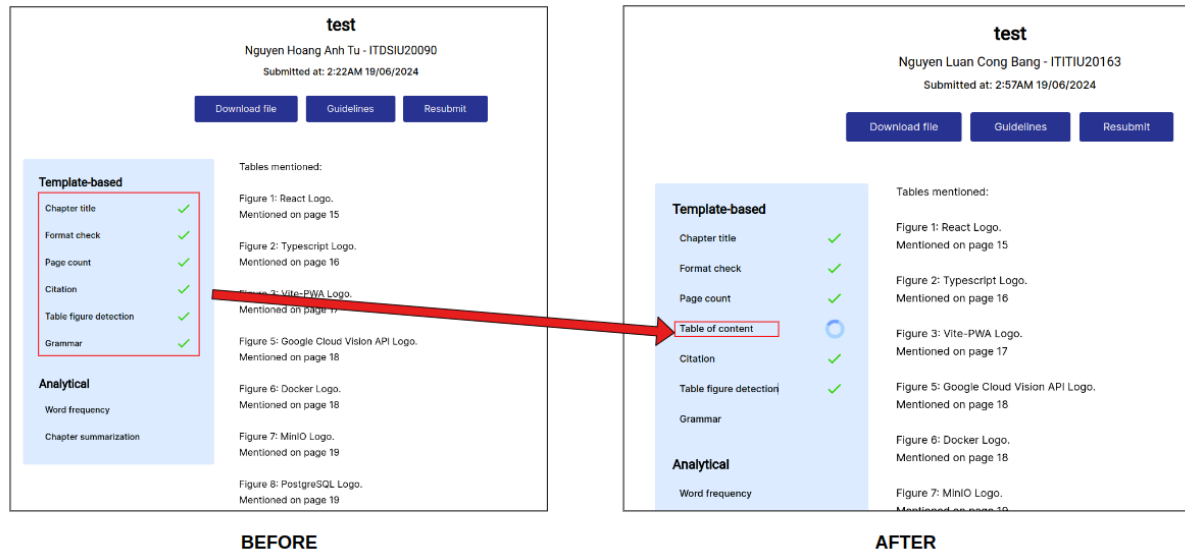


Figure 51. Example of adding Table of content service

For example, suppose a new service appears in the table of contents in Figure 51. In that case, the CI will automatically detect this addition and trigger a process to update the manifest deployment file, thus including the new service during deployment.

```
[Pipeline] readFile
[Pipeline] echo
Added service: chapter-summarization
[Pipeline] echo
Added service: chapter-title
[Pipeline] echo
Added service: page-count
[Pipeline] echo
Added service: citation
[Pipeline] echo
Added service: table-figure-detection
[Pipeline] echo
Added service: grammar
[Pipeline] echo
Added service: format-check
[Pipeline] echo
Added service: word-frequency
[Pipeline] echo
Added service: table-of-content
[Pipeline] writeFile
[Pipeline] writeFile
```

Figure 52. Adding service in CD pipeline

Furthermore, for organized tracking, the names of added services will be recorded in a file named *services.txt*, as shown in Figure 52.

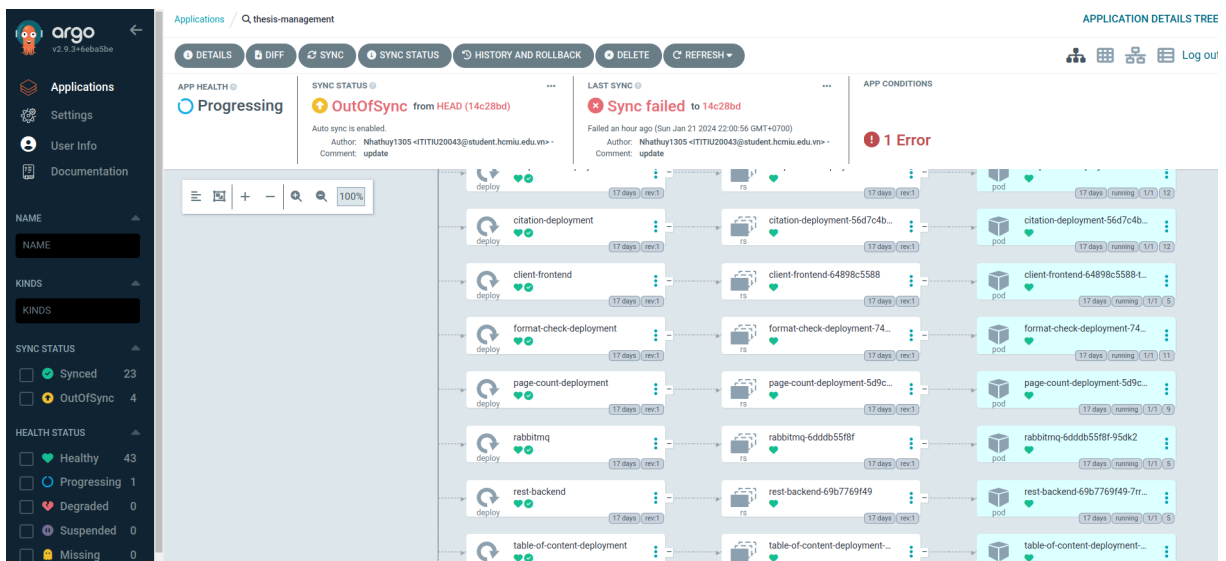


Figure 53. ArgoCD automates update Cluster

ArgoCD, positioned at the intersection of deployment and code management, vigilantly monitors the repository for changes. When a modification is committed, ArgoCD automatically initiates an update process to synchronize the running application with the latest changes, as depicted in Figure 53. This automated oversight guarantees that the Thesis Report Management System consistently aligns with the most recent code developments, maintaining up-to-date functionality, as illustrated in Figure 54.

Name	Namespace	Containers	CPU	Memory	Restarts	Controlled By	Node	QoS	Age	Status
chapter-summarization-deployment-6bb65b5bc8-9k5xf	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	30m	Running
chapter-title-deployment-6c97f965d6-49trx	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	10m	Running
citation-deployment-5d5ddcb854-h7rcr	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	30m	Running
client-frontend-deployment-65dd949b66-l2ck9	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	30m	Running
format-check-deployment-6b59786b76-z4744	default	1	N/A	N/A	2	ReplicaSet	node2	BestEffort	30m	Running
grammar-deployment-699dc8fd64-2q2kv	default	1	N/A	N/A	2	ReplicaSet	node2	BestEffort	30m	Running
page-count-deployment-7cc7b77f4b-v458k	default	1	N/A	N/A	2	ReplicaSet	node3	BestEffort	30m	Running
postgresql-0	default	1	N/A	N/A	0	StatefulSet	node3	BestEffort	30m	Running
rabbitmq-656c5f77cb-tc6wk	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	30m	Running
rest-backend-deployment-6b5849bb74-5b9nm	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	29m	Running
table-figure-detection-deployment-6f6f5f4ff-82zgw	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	30m	Running
table-of-content-deployment-86d6b0bd58-nzbzq	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	9m56s	Running
word-frequency-deployment-7fb4b54f5f-6gxmm	default	1	N/A	N/A	2	ReplicaSet	node3	BestEffort	30m	Running

Figure 54. Pods change on the Kubernetes cluster

The TRMS benefits from ArgoCD's commitment to maintaining consistency across different environments, from the security of a local system to the broader scope of a VM server. Serving as a conduit between deployment and coding, ArgoCD ensures that each update is smoothly integrated into both local and cloud-based environments.

Name	Namespace	Type	Cluster IP	Ports	External IP	Selector	Age	Status
chapter-summarization-service	default	ClusterIP	10.233.61.133	6002/TCP	-	app=chapte	6d23h	Active
chapter-title-service	default	ClusterIP	10.233.58.100	6003/TCP	-	app=chapte	6d23h	Active
citation-service	default	ClusterIP	10.233.19.136	6006/TCP	-	app=citator	6d23h	Active
client-frontend-service	default	NodePort	10.233.25.15	3000:30005/TCP	-	app=client-f	4d21h	Active
format-check-service	default	ClusterIP	10.233.42.231	6009/TCP	-	app=format	6d23h	Active
grammar-service	default	ClusterIP	10.233.27.162	6008/TCP	-	app=gramm	6d23h	Active
kubernetes	default	ClusterIP	10.233.0.1	443:6443/TCP	-		21d	Active
page-count-service	default	ClusterIP	10.233.27.86	6004/TCP	-	app=page-c	6d23h	Active
postgresql	default	ClusterIP	10.233.54.245	5432/TCP	-	app=postgrn	6d23h	Active
rabbitmq	default	ClusterIP	10.233.42.155	5672/TCP	-	app=rabbitr	6d23h	Active
rest-service	default	NodePort	10.233.29.70	5000:30010/TCP	-	app=rest-ba	6d23h	Active
table-figure-detection-service	default	ClusterIP	10.233.6.196	6007/TCP	-	app=table-fi	6d23h	Active
table-of-content-service	default	ClusterIP	10.233.49.66	6005/TCP	-	app=table-o	6d23h	Active
word-frequency-service	default	ClusterIP	10.233.54.168	6010/TCP	-	app=word-fi	2d10h	Active

Figure 55. Services exposing a network for each pod

Services ensure the network interface remains consistent and accessible even as individual pods are created, updated, or deleted. This abstraction allows clients within and outside the cluster to communicate with pods under a single addressable endpoint, promoting seamless interaction and connectivity without tracking each pod's internal IP changes.

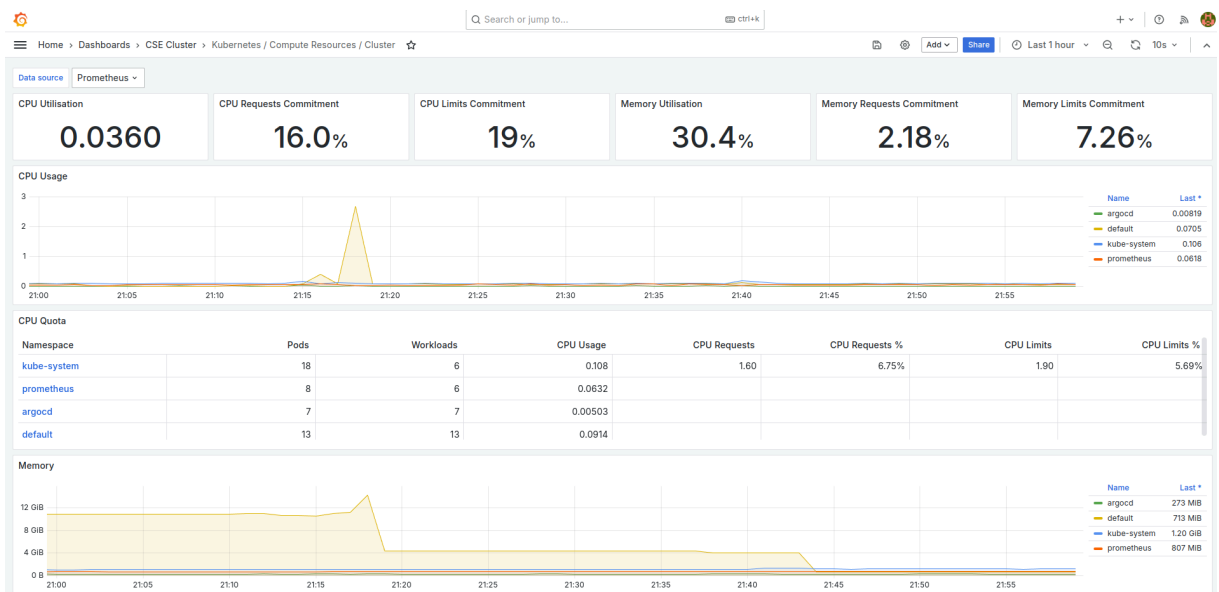


Figure 56. Grafana dashboard

Finally, we can access the Grafana dashboard, as shown in Figure 56. It is designed to provide users with a comprehensive overview of cluster metrics and data points, which can be crucial for monitoring system health, analyzing application performance, and identifying anomalies.

CHAPTER 5

DISCUSSION AND EVALUATION

5.1. Discussion

The method brainstormed in our dissertation, event-driven architecture along with Python, to configure a system for managing thesis administration is the focus of the thesis. This was a mimetic ideal for a CI/CD system. It was very flexible in terms of transformations, faults, and rearrangements. We sit in the great conjunction of several stars. We also involved DevOps methods, as well as some NLP techniques, in purifying the action.

The first thing achieved was automatizing repetitive activities, including the summary of documents, the division of chapters, and citation tracking. This makes the automation of these tasks possible not only efficient but also generates other advantages both for students and a supervisor, such as the development of a more terrific work environment, the growth of efficiency and productivity, and a constant increase in the quality of the thesis. Even though we had some limitations in evaluating the NLP techniques and tools, the resting research can explore the areas and broaden the system's impact on the workflows.

As a final result, our study showed that using this architecture and NLP would significantly enhance thesis management and make the research process more effective for all participants.

5.2. Comparison

Current tools like thesis report management systems (TRMS) and research management software can only store documents, collaborate, and manage references. They are not perfectly satisfactory in the thesis work area. Present natural language processing (NLP)--based thesis analysis projects, while they certainly contribute to some areas of the world, cannot cover a wide range of functions such as our system does, for example, plagiarism detection.

One component of our system that makes it unique is the combination of event-driven architecture and NLP into one product. The event-driven approach ensures items, such as modules, can be replaced among other independent parts during a task. When this happens, the process continues to operate horizontally, error-free, and the data flows nicely and easily from one module to another. This, in turn, leads to a more efficient workflow, where researchers have more time to focus on the actual content of their work diligently.

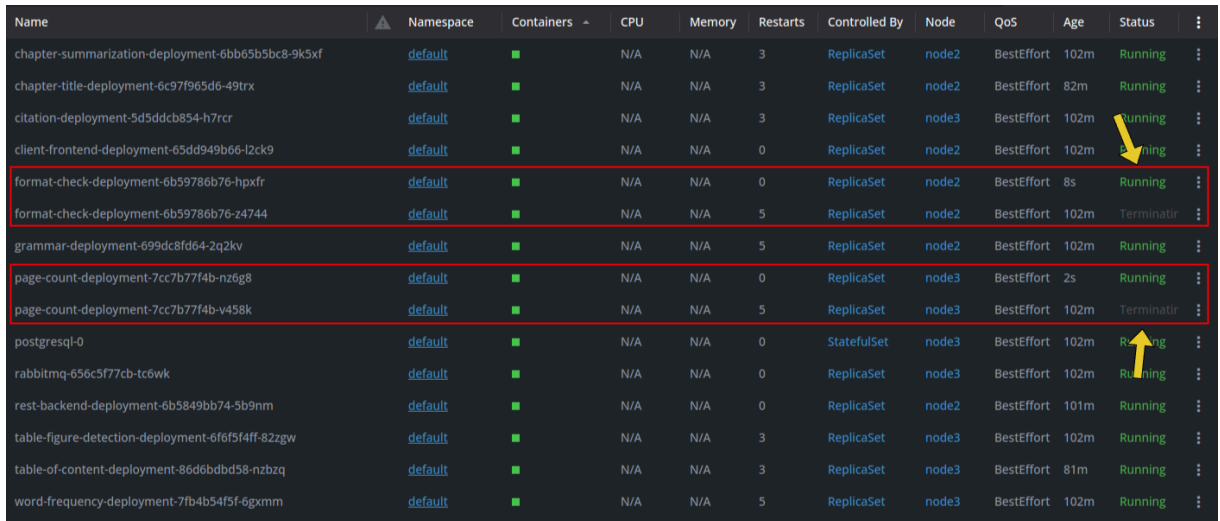
My system has been developed using the most recent NLP techniques, which means it does more than check the text spelling and grammar. It also identifies and underscores the possible design mistakes and structural nonsense, the most common errors a researcher can make. Technically, we won't make a statement often enough that the system can produce the results that the other system delivers. However, when the program is carried out, and the data is processed, it shows how the different tools and more advanced tools can work.

In the final analysis, our thesis paper will depict the creation of a great application and how we build it all. It will be accomplished by combining cutting-edge technology with a deep understanding of research needs in the form of theses so that the process is no longer as annoying as it is sometimes but much more like a leisurely walking journey through a park.

5.3. Evaluation

5.3.1. Pods fail while executing

We will assess a Kubernetes cluster's resilience and fault tolerance by deleting pods and deliberately observing the system's response. This evaluation involves terminating each pod within the cluster and monitoring how quickly and efficiently Kubernetes schedules and deploys replacement pods with different identifiers.



Name	Namespace	Containers	CPU	Memory	Restarts	Controlled By	Node	QoS	Age	Status
chapter-summarization-deployment-6bb65b5bc8-9k5xf	default	1	N/A	N/A	3	ReplicaSet	node2	BestEffort	102m	Running
chapter-title-deployment-6c97f965d6-49trx	default	1	N/A	N/A	3	ReplicaSet	node2	BestEffort	82m	Running
citation-deployment-5d5ddcb854-h7rcr	default	1	N/A	N/A	3	ReplicaSet	node3	BestEffort	102m	Running
client-frontend-deployment-65dd949b66-l2ck9	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	102m	Running
format-check-deployment-6b59786b76-hpxfr	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	8s	Running
format-check-deployment-6b59786b76-z4744	default	1	N/A	N/A	5	ReplicaSet	node2	BestEffort	102m	Terminating
grammar-deployment-699dc8fd64-2q2kv	default	1	N/A	N/A	5	ReplicaSet	node2	BestEffort	102m	Running
page-count-deployment-7cc7b77f4b-nz6g8	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	2s	Running
page-count-deployment-7cc7b77f4b-v458k	default	1	N/A	N/A	5	ReplicaSet	node3	BestEffort	102m	Terminating
postgresql-0	default	1	N/A	N/A	0	StatefulSet	node3	BestEffort	102m	Running
rabbitmq-656c5f77cb-tc6wk	default	1	N/A	N/A	0	ReplicaSet	node3	BestEffort	102m	Running
rest-backend-deployment-6b5849bb74-5b9nm	default	1	N/A	N/A	0	ReplicaSet	node2	BestEffort	101m	Running
table-figure-detection-deployment-6f6f54ff-82zgw	default	1	N/A	N/A	3	ReplicaSet	node3	BestEffort	102m	Running
table-of-content-deployment-86d6dbd58-nzbzq	default	1	N/A	N/A	3	ReplicaSet	node3	BestEffort	81m	Running
word-frequency-deployment-7fb4b54f5f-6gxmm	default	1	N/A	N/A	5	ReplicaSet	node3	BestEffort	102m	Running

Figure 57. New pods created right after the current pods deleted

As a result, it was observed that the replacement pods were immediately created within 1-2 seconds after the current pods were deleted, as illustrated in Figure 57.

To sum up, the cluster's ability to rapidly recover from potential failures ensures reliability and supports continuous service delivery, making it an exemplary model of resilience in dynamic computing environments.

5.3.2. Performance testing

To conduct a rigorous benchmark of request handling capabilities across specific nodes within a Kubernetes cluster. We will utilize *hey*, a renowned HTTP load generator and a modern alternative to ApacheBench (ab), to administer a series of controlled tests. We aim to uncover any disparities in load handling and response times by methodically increasing the load and monitoring how different nodes process requests:

- First benchmark: Send 100 requests with 100 concurrent users:

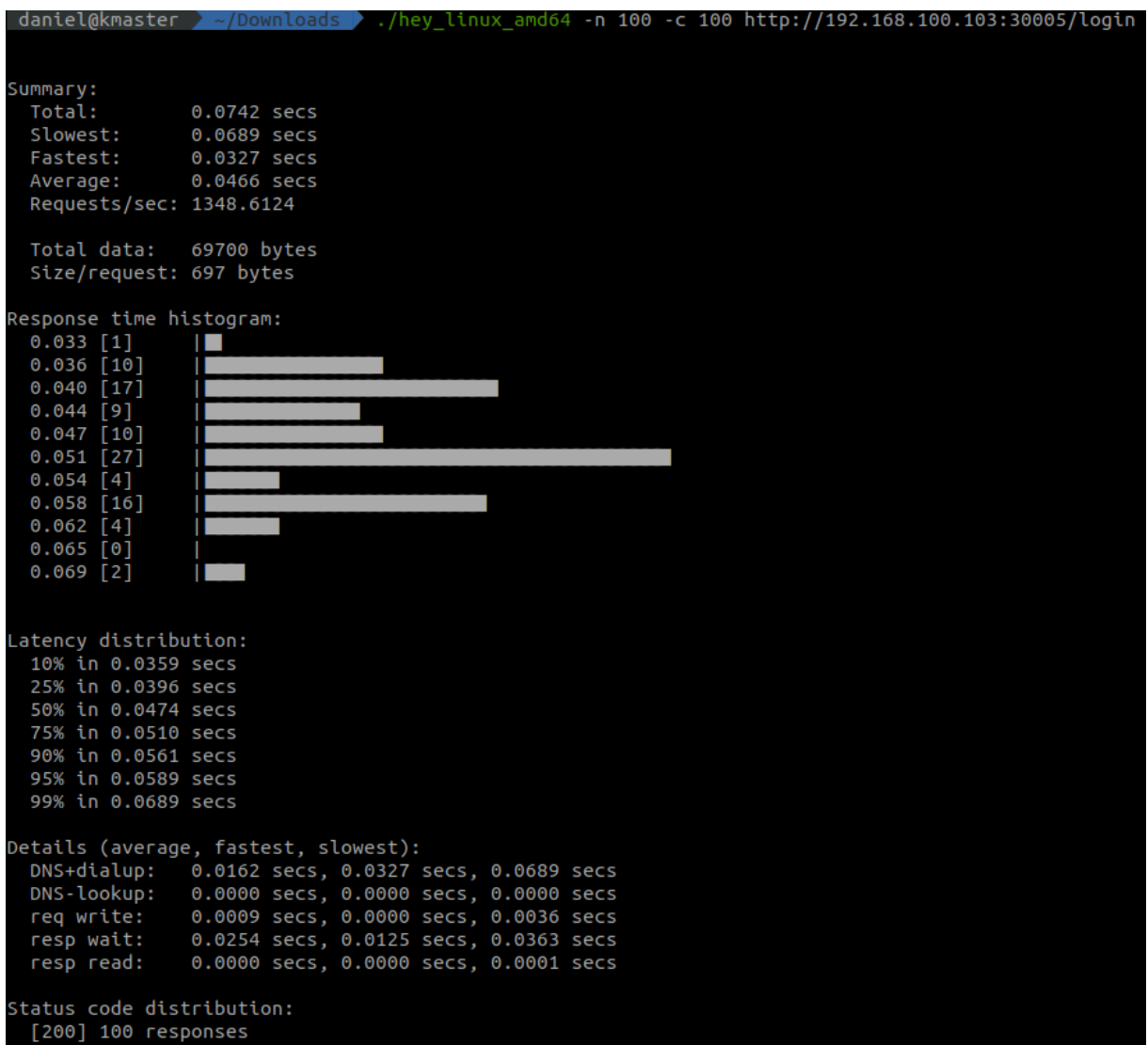


Figure 58. First benchmark test with 100 users

The initial benchmark test conducted on a node within the Kubernetes cluster has yielded positive results, indicating robust performance under a simulated load of 100 concurrent requests. The test was completed in just over 0.07 seconds, with an impressive throughput of approximately 1348 requests per second. Response times were generally quick,

averaging around 0.0466 seconds, demonstrating the node's ability to handle high traffic efficiently.

- Second benchmark: Send 1000 requests with 100 concurrent users:

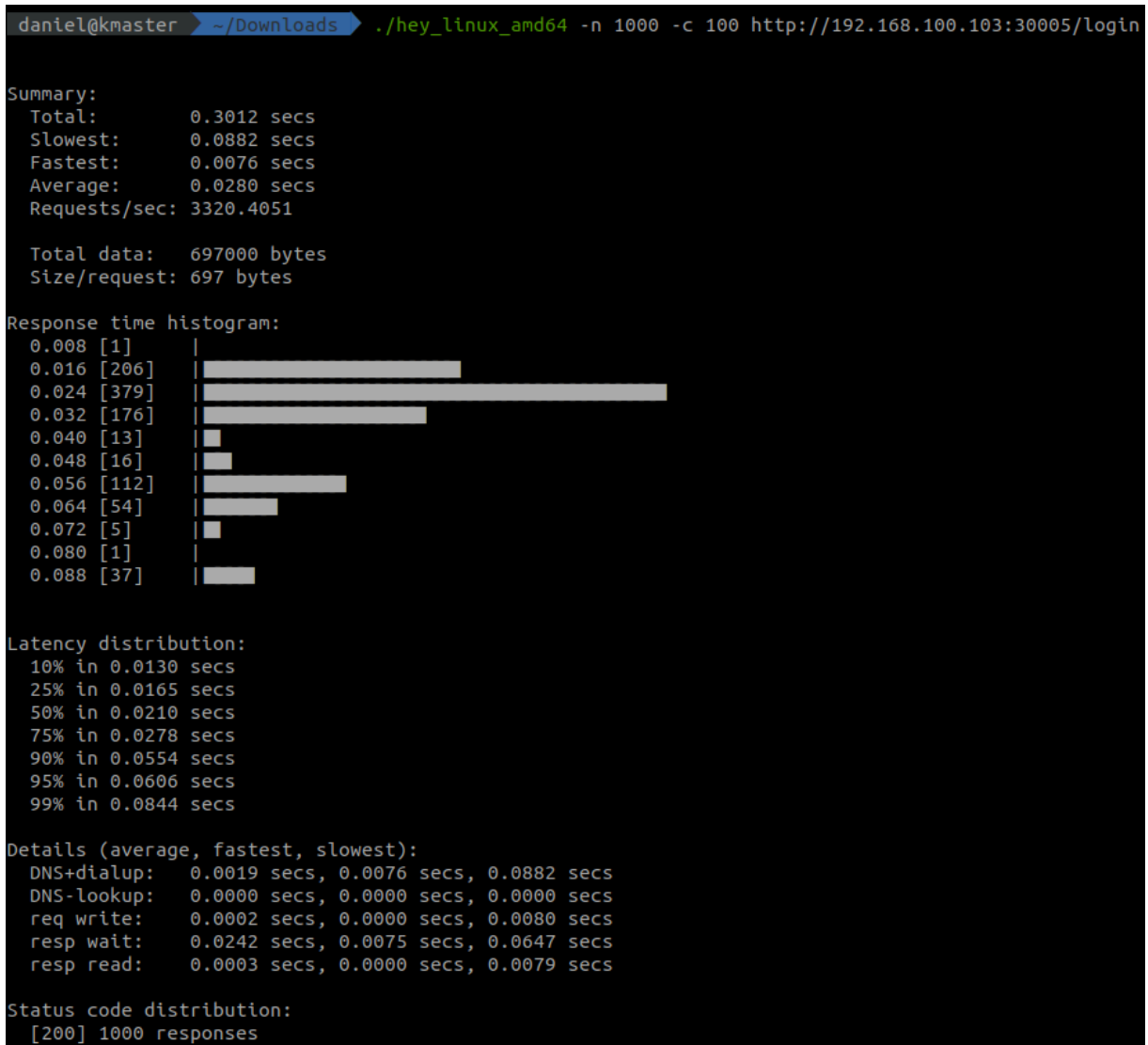


Figure 59. Second benchmark test with 1000 users

The second benchmark test was significantly scaled to handle 1000 requests with 100 concurrent users and performed on the same Kubernetes node. This test also showed promising results, completing in just over 0.3 seconds. The node managed an excellent throughput rate of 3320.4051 requests per second, underscoring its capability to efficiently process high volumes of traffic.

- Third benchmark: Send 100000 requests with 100 concurrent users:

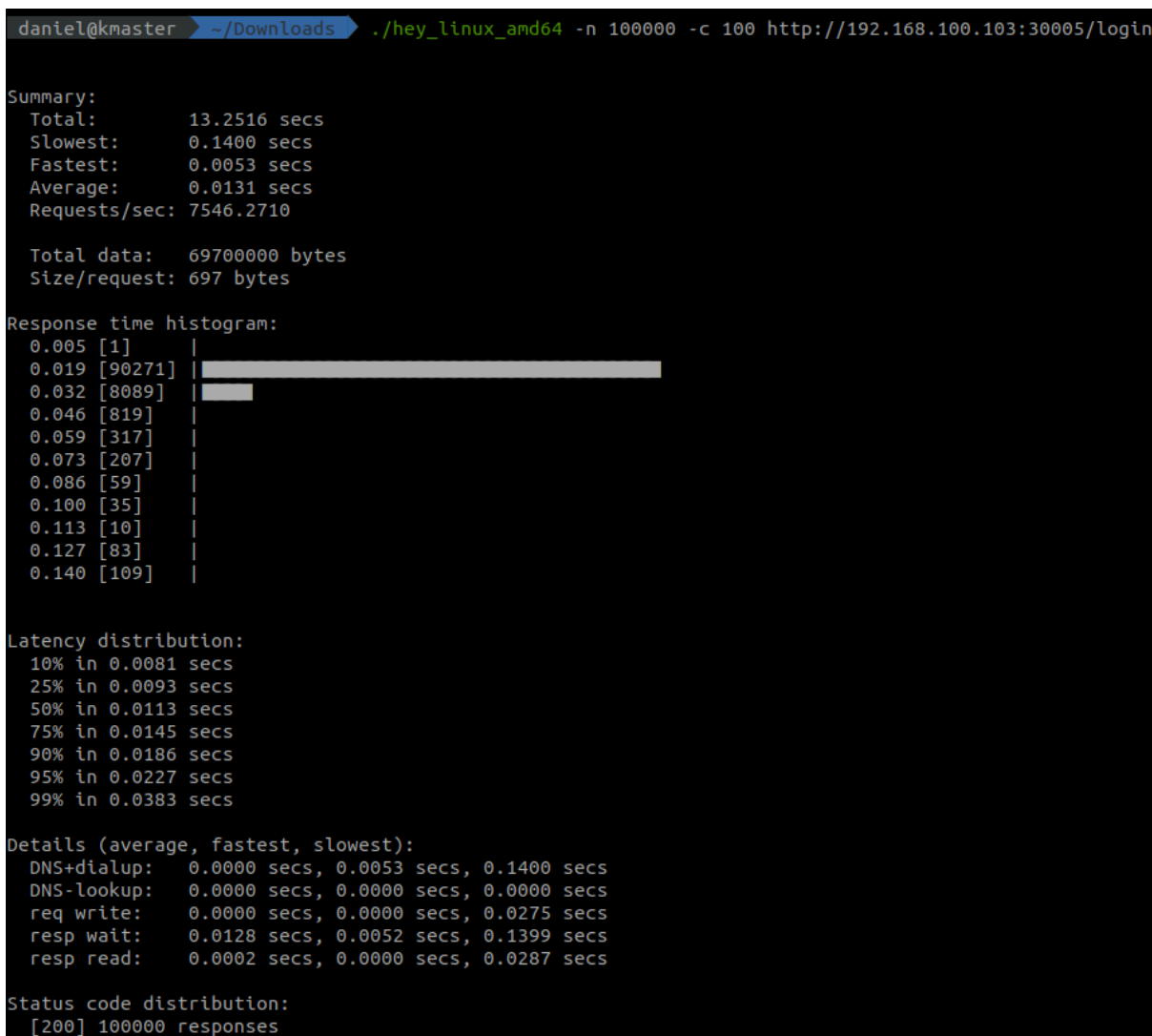


Figure 60. Third benchmark test with 100000 users

The third benchmark test dramatically increased the load to 100,000 requests with 100 concurrent users on the same Kubernetes node, providing further insights into the system's performance under extreme conditions. This extensive test was completed in 13.2516 seconds, demonstrating the node's remarkable ability to handle high request volumes efficiently.

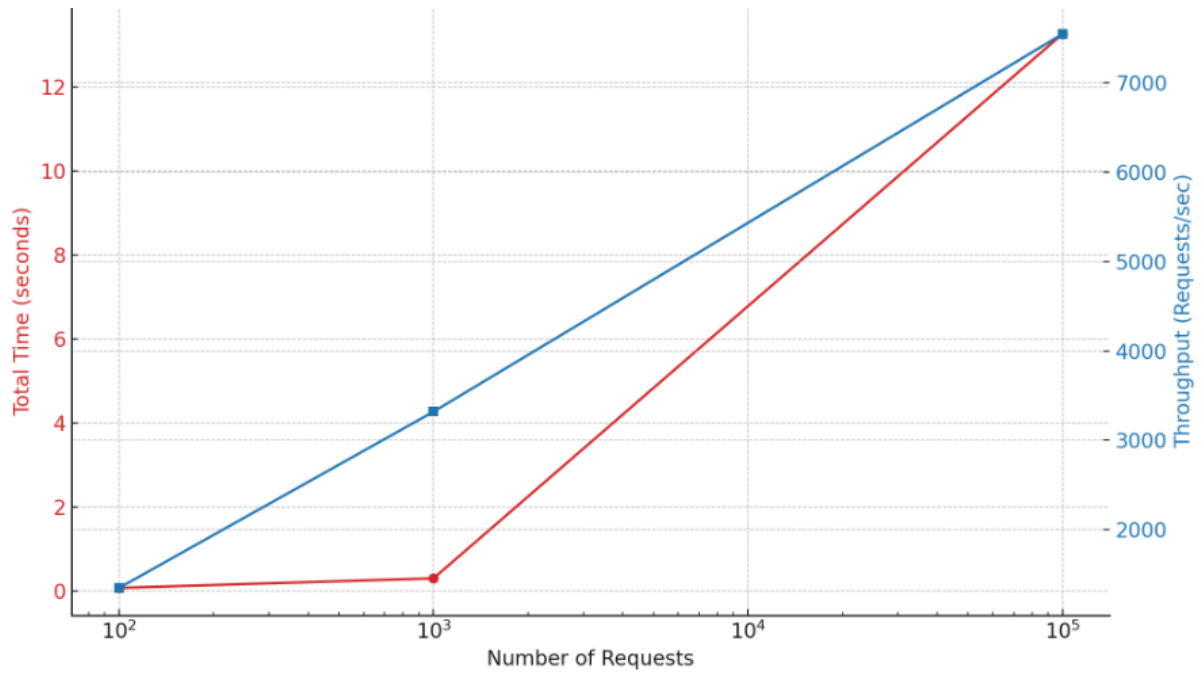


Figure 61. Kubernetes Node Benchmark: Time and Throughput

The chart above illustrates the performance of the Kubernetes node across three different tests, comparing the total response time and throughput as the number of requests increases from 100 to 100,000. As observed, while the total time increases with the number of requests, the throughput also substantially rises, indicating the node's effective scaling and handling capabilities under varying loads.

5.3.3. Real-life evaluation

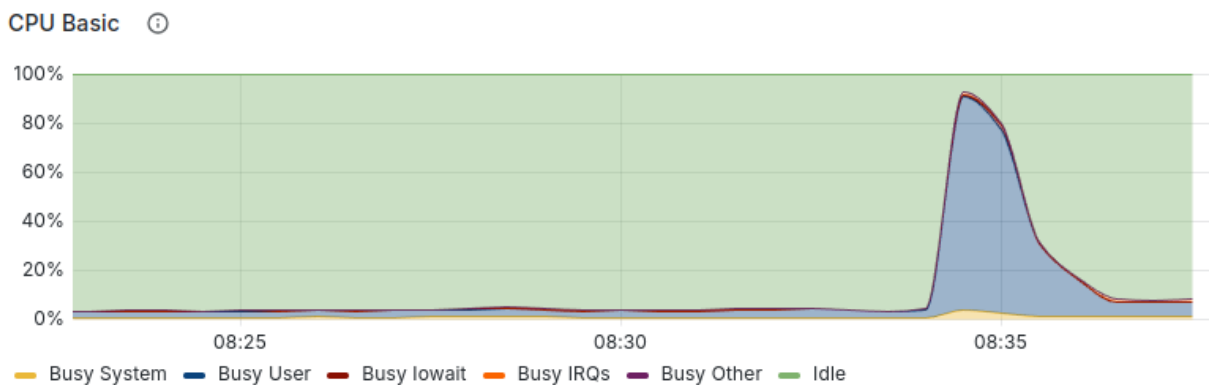


Figure 62. Percentage CPU usage on a worker node handling 10 reports at the same time

We performed on all the Python services, precisely its upload functionality in this part. We sent ten reports from each user to the system. As observed from the Grafana dashboard in Figure 62, the CPU utilization of node 3 (worker 2) peaked at approximately 97.3%. The controller node was crucial in this scenario, facilitating data distribution between workers one and 2. This spike in CPU usage, depicted in the graph, reflects the system's response to the

increased workload, underscoring the effectiveness of the controller node in managing and balancing the load between the worker nodes.

Thread Properties

Number of Threads (users):

100

Ramp-up period (seconds):

1

Loop Count:

☐ Infinite

1

☒ Same user on each iteration

Figure 63. Configure simultaneous requests in jMeter

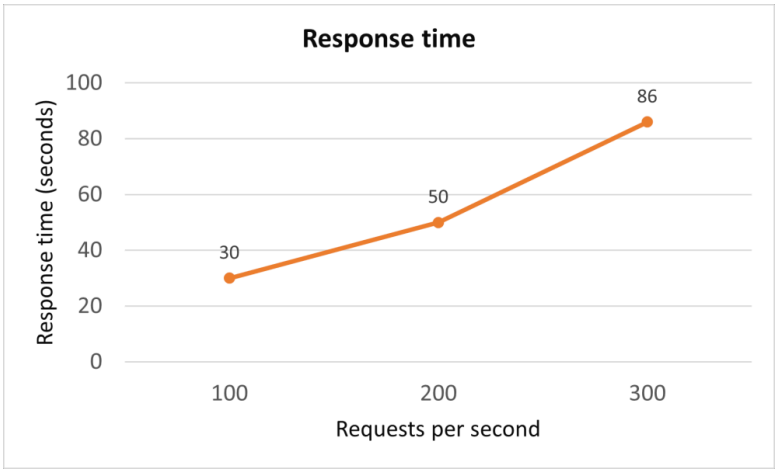


Figure 64. Response time vs Request per second

As we can see, the upload capability works relatively smoothly under 300 requests. However, issues appear when 400 requests are tested, indicating a bottleneck. The presumptions state that at least ten submissions of these will be sent out daily. The load test demonstrates that this application has the capability to hold the contributions before uploading them to the cloud storage and that it can respond quickly enough to meet that demand.

```
List of Citations:
Correct Citations:
[1] TradingView PineScript, https://tradingview.com/pine-script-docs/en/v5/Introduction.html
(Type: IEEE Style (Website))
[2] Node Puppeteer, https://pptr.dev/ (Type: IEEE Style (Website))
No issues found in citations.

Grade: 100%
Service Result: Pass

-----
Page count required for each chapter:
TOTAL: 40
CHAPTER 1: 8
CHAPTER 2: 5
CHAPTER 3: 10
CHAPTER 4: 10
CHAPTER 5: 3
CHAPTER 6: 3
REFERENCES: 1

Page count detected in document:
TOTAL: 35
CHAPTER 1: 3
CHAPTER 2: 5
CHAPTER 3: 4
CHAPTER 4: 10
CHAPTER 5: 3
CHAPTER 6: 1
REFERENCES: 1

Comparison:
TOTAL: Not enough
CHAPTER 1: Not enough
CHAPTER 2: Exceeded
```

Figure 65. Sample PDF report

For each user, the system successfully generated and returned a comprehensive PDF report detailing the results of all services evaluated, including their grades and status (pass or fail). This automated report provides users with a clear and detailed overview of their performance metrics, effectively summarizing the outcomes of the tests conducted on the system.

5.3.4. Conclusion

Because of Kubernetes' resilience, the system has shown its ability to scale, balance load, and maintain high availability via several tests. Notwithstanding the system's flaws, it demonstrated its robustness and efficiency by successfully processing multiple user requests simultaneously and generating detailed PDF reports for each user. Also, the controller node's role in efficiently distributing tasks between worker nodes ensured smooth operation and timely responses.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1. Conclusion

The Thesis Report Management System (TRMS) originated as a modest project and has since developed into a comprehensive cloud-based suite of functionalities managed by Kubernetes and automated by ArgoCD. This strategic transition to a less conventional approach has facilitated the development of innovative features and significantly transformed the operational landscape, catalyzing a shift towards greater agility in academic environments.

The progress of these technologies exceeds the accomplishment of technological milestones. The TRMS brings several new capabilities to researchers:

- The democratization of Access: By abolishing institutional limitations, the cloud platform facilitates supercomputing capabilities for researchers from all corners of the globe. Not only does it help level the playing field and take knowledge democratization to a new level but it also becomes the propeller that speeds up global research.
- Instant Collaboration: Sharing data, ideas, and analyses become the norm. The TRMS is the cradle of a rosy research community where the collaborative inquiry of a wide array of research members begins, breaks, and extends through geographical limitations.
- Faster Innovation: Robot acting mimics complex computing applications; thus, the freemen of mind can follow their innermost interests. Escaped from everyday work, scientists can dabble in new areas, take dangerous theories, and use revolutionary ideas.

The TRMS is the ultimate proof that technology can be a force of change in higher education. Automation, ArgoCD, and Kubernetes accomplished complete, smooth, sleek, and effective integration in the research so that the researchers are no longer challenged in their knowledge journey. They can experience it with grace and agility. It functions more than just as an instrument but also as an initiation into a newly emerged culture of unbreakable research unity, fueled by discovery, driven forward by invention, and continually pushing against the boundaries of knowledge. The TRMS has introduced a new period of research agility, marked

by knowledge quest and exploration without the fear of limits and curiosity run without restrictions.

6.2. Future work

The Thesis Report Management System (TRMS) is well on its way to completion, although its path is not smooth. Although the current development of theses through AI-based personalization and the automation of some tasks is marked by the success of thesis management, the highest degree of potential of the system lies across all fields of knowledge. My commitment to innovation makes it our priority to take the TRMS beyond the dimension of standard IT support and allow it to be dynamic, which not only serves different departments such as Business Administration, Biomedical Engineering, Logistics and Supply Chain Management, and Civil Engineering, etc. but is also supplemented with customizable features for each field. This possibility of customization allows TRMS to work with the university's IT department and offer a variety of template options targeting all departments in the university and the staff who need them.

REFERENCES

- [1] J. Swerzenski, "Critically analyzing the online classroom: Blackboard, Moodle, Canvas, and the pedagogy they produce," *J. Commun. Pedagogy*, vol. 4, pp. 51–69, Jan. 2021.
- [2] D. P. Nolasco and D. M. P. Hernandez, "Learning Management System usage: Unwrapping its potentials and drawbacks," *CTU J. Innov. Sustain. Dev.*, vol. 15, no. 2, pp. 92–102, Jul. 2023, doi: 10.22144/ctu.jen.2023.024.
- [3] M. Y. M. Chong, "A study on plagiarism detection and plagiarism direction identification using natural language processing techniques." University of Wolverhampton, 2013. [Online]. Available: <http://hdl.handle.net/2436/298219>
- [4] M. Senapathi, J. Buchan, and H. Osman, "DevOps Capabilities, Practices, and Challenges: Insights from a Case Study," in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, in EASE '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 57–67. doi: 10.1145/3210459.3210465.
- [5] A. Homayouni, "NLP-based Failure log Clustering to Enable Batch Log Processing in Industrial DevOps Setting," Student thesis, 2022. Accessed: Jun. 20, 2022. [Online]. Available: <http://urn.kb.se/resolve?urn=urn:nbn:se:mdh:diva-59203>
- [6] S. Mansfield-Devine, "DevOps: finding room for security," *Netw. Secur.*, vol. 2018, no. 7, pp. 15–20, Jul. 2018, doi: 10.1016/S1353-4858(18)30070-9.
- [7] D. Weaver, C. Spratt, and C. S. Nair, "Academic and student use of a learning management system: Implications for quality," *Australas. J. Educ. Technol.*, vol. 24, no. 1, Jan. 2008, doi: 10.14742/ajet.1228.
- [8] S. Panda, "Enhancing PDF interaction for a more engaging user experience in library: Introducing ChatPDF," *IP Indian J. Libr. Sci. Inf. Technol.*, vol. 8, no. 1, pp. 20–25, Jun. 2023, doi: 10.18231/j.ijlsit.2023.004.
- [9] T. Prem Jacob, B. L. S. Bizotto, and M. Sathiyarayanan, "Constructing the ChatGPT for PDF Files with Langchain – AI," in *2024 International Conference on Inventive Computation Technologies (ICICT)*, Lalitpur, Nepal: IEEE, Apr. 2024, pp. 835–839. doi: 10.1109/ICICT60155.2024.10544643.
- [10] M. Firat, "How Chat GPT Can Transform Autodidactic Experiences and Open Education?," Open Science Framework, preprint, Jan. 2023. doi: 10.31219/osf.io/9ge8m.
- [11] N. Petrović, "Chat GPT-Based Design-Time DevSecOps," in *2023 58th International Scientific Conference on Information, Communication and Energy Systems and*

- Technologies (ICEST)*, Nis, Serbia: IEEE, Jun. 2023, pp. 143–146. doi: 10.1109/ICEST58410.2023.10187247.
- [12] V. Plevris, G. Papazafeiropoulos, and A. Jiménez Rios, “Chatbots Put to the Test in Math and Logic Problems: A Comparison and Assessment of ChatGPT-3.5, ChatGPT-4, and Google Bard,” *AI*, vol. 4, no. 4, pp. 949–969, 2023, doi: 10.3390/ai4040048.
- [13] S.-L. Cheng *et al.*, “Comparisons of Quality, Correctness, and Similarity Between ChatGPT-Generated and Human-Written Abstracts for Basic Research: Cross-Sectional Study,” *J Med Internet Res*, vol. 25, p. e51229, Dec. 2023, doi: 10.2196/51229.
- [14] N. Meuschke, A. Jagdale, T. Spinde, J. Mitrović, and B. Gipp, “A Benchmark of PDF Information Extraction Tools Using a Multi-task and Multi-domain Evaluation Framework for Academic Documents,” in *Information for a Better World: Normality, Virtuality, Physicality, Inclusivity*, vol. 13972, I. Sserwanga, A. Goulding, H. Moulaison-Sandy, J. T. Du, A. L. Soares, V. Hessami, and R. D. Frank, Eds., in Lecture Notes in Computer Science, vol. 13972. , Cham: Springer Nature Switzerland, 2023, pp. 383–405. doi: 10.1007/978-3-031-28032-0_31.

APPENDIX